“시간이 아깝지 않은”

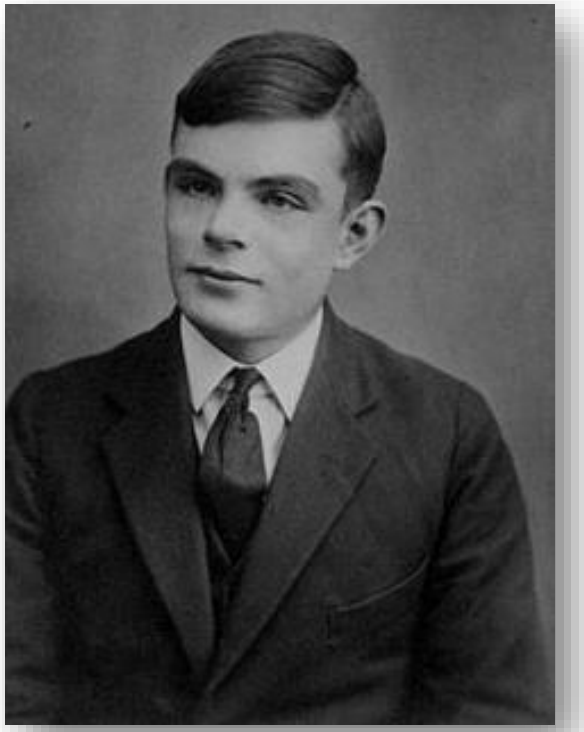
SK네트웍스 Family AI캠프 교재6

강사 이영배

09. 머신 러닝

- 기본 개념

인공지능의 역사



Alan Mathison Turing(앨런 매티슨 튜링)
(1912.6.23 ~ 1954.6.7)
영국의 수학자,암호학자,논리학자

데이터 분석 및 모델링 과정

- 데이터 분석 과정

1) 데이터 수집 : 분석에 필요한 데이터 확보

2) 데이터 전처리(preprocessing) : 데이터를 분석에 적합한 형태로 만들어서 신뢰도를 높이는 작업

3) 데이터 탐색 : 그래프나 통계적인 방법을 이용, 데이터의 특징과 패턴을 파악

4) 모델링 : 수집 • 정제된 데이터를 기반으로 수학적 • 통계적인 방법을 사용하여 데이터 간의 관계를 파악하고, 새로운 데이터에 대해서 결과를 예측하거나 설명할 수 있는 규칙을 만드는 과정

5) 분석 모델 평가

인공지능과 머신 러닝



sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
7.0	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6.0	2.5	virginica
5.8	2.7	5.1	1.9	virginica
7.1	3.0	5.9	2.1	virginica



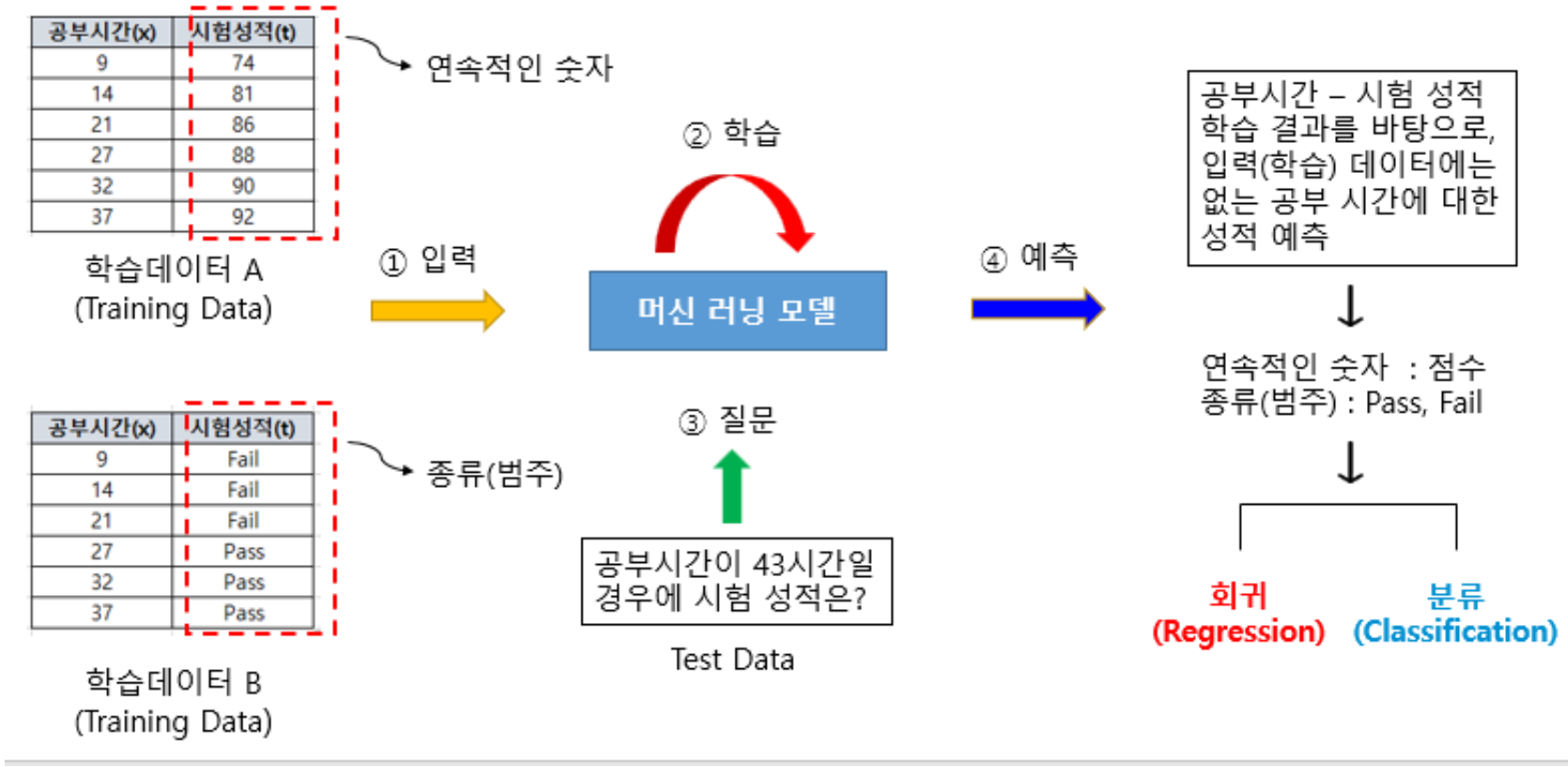
머신 러닝 종류(기준 : 학습 방식)

- 지도 학습(Supervised Learning)
- 비지도 학습(Unsupervised Learning)

지도 학습(Supervised Learning)

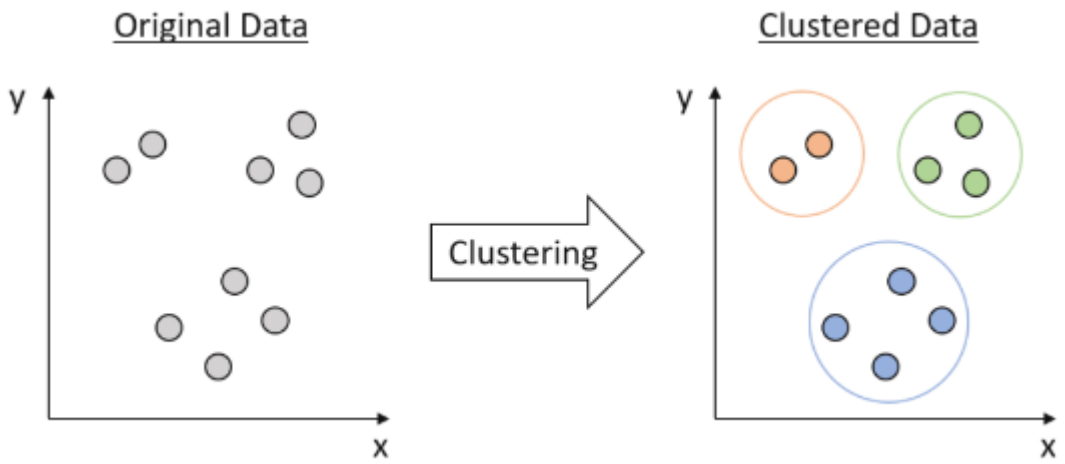
- 데이터에 대한 '정답'이 주어진 상태에서 머신러닝 모델을 학습시키는 방법
- 분야(task)
  1) 분류 : 어떤 '범주'에 속할 지를 예측 예) 품종 분류, 종양 분류(양성, 악성)
  2) 회귀 : 숫자 값을 예측 예) 수량, 가격 예측

지도 학습(Supervised Learning)



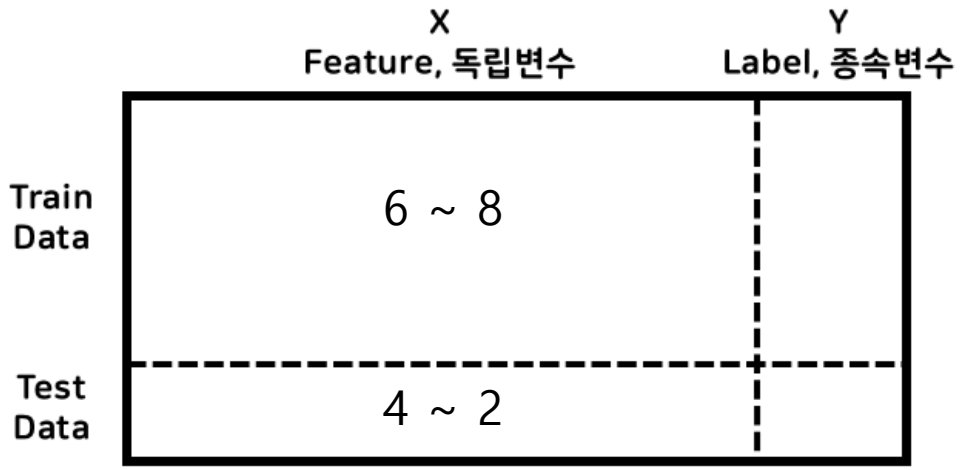
비지도 학습(Unsupervised Learning)

- 데이터에 대한 '정답'이 없는 상태에서 머신 러닝 모델을 학습시키는 방법
- 데이터를 비슷한 특성끼리 묶는 클러스터링(Clustering) 등이 있음



머신러닝 학습 – 데이터의 분할

- 학습용 데이터와 평가용 데이터로 분할



모델 평가

- 미지의 데이터 ⇒ 결과(범주 또는 값) 예측
- 평가 척도
  ① accuracy(정확도)
  ② recall(재현율)
  ③ precision(정밀도)
  ④ f1 score
  ⑤ MSE / RMSE

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

1) 개념: 분석가의 전문 지식과 통찰력을 데이터에 직접 주입하여, 머신러닝 모델이 더 똑똑한 예측을 하도록 만드는 과정

① 파생 변수 생성 (Feature Creation)

- 개념: 기존에 있던 변수(데이터 컬럼)들을 조합하여 새롭고 더 강력한 의미를 가진 변수를 만드는 것
- 예시: 광고비와 매출 변수로 광고비 대비 수익률(ROAS)이라는 분석에 유의미한 컬럼을 생성

② 범주형 변수 변환 (Encoding)

- 개념: '성별', '채널', '연령대'와 같이 글자로 된 데이터를 모델이 이해할 수 있는 숫자로 변환하는 작업
- 방법: 원-핫 인코딩 (One-Hot Encoding), Label Encoding

③ 연속형 변수 변환 (Scaling)

- 개념: 구매액, 방문횟수와 같이 숫자로 된 데이터의 분포나 범위를 조정하여 모델의 성능을 높이는 작업
- 방법: 표준화(Standardization), 정규화(Normalization)

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

2) One-Hot Encoding

① 개념: Channel 컬럼의 'Google', 'Facebook', 'Naver' 값을 각각 [1 0 0], [0 1 0], [0 0 1]과 같이 고유한 숫자 컬럼으로 만들어주는 방법

- 범주형 컬럼의 문자열의 고유한 값 (n개) → 새로운 컬럼 생성 (n개)
- 기존 범주형 컬럼의 각 문자열 → 새로운 컬럼 중 해당 문자열 컬럼에 1, 나머지 컬럼에 0을 할당

id	day
0	월
1	월
2	화
3	수
4	화



id	day_월	day_화	day_수
0	1	0	0
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

2) One-Hot Encoding

② 구현 방법

- pd.get_dummies(data=df, dtype=int)
- pd.get_dummies(data=df, columns=['col1', 'col2', ..], dtype=int)

	name	color
0	apple	red
1	banana	yellow
2	cherry	red
3	durian	green
4	NaN	NaN

```
pd.get_dummies(fruit)   #dataframe 전체를 했을 때
```

	name_apple	name_banana	name_cherry	name_durian	color_green	color_red	color_yellow
0	1	0	0	0	0	1	0
1	0	1	0	0	0	0	1
2	0	0	1	0	0	1	0
3	0	0	0	1	1	0	0
4	0	0	0	0	0	0	0

```
pd.get_dummies(fruit, columns=['name'])   #dataframe에서 특정 열만 인코딩
```

	color	name_apple	name_banana	name_cherry	name_durian
0	red	1	0	0	0
1	yellow	0	1	0	0
2	red	0	0	1	0
3	green	0	0	0	1
4	NaN	0	0	0	0

One-Hot Encoding 실습

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

3) Label Encoding

① 개념 : 각 컬럼의 범주형 문자열 데이터를 0부터 n-1까지의 연속적 수치 데이터로 표현하는 것

② 구현 방법

- df.replace({'문자열1':숫자1, 문자열2':숫자2, ...})

  예시) df = df.replace({'no': 0, 'yes': 1})



	age	job	marital	education	default	balance	housing	loan
0	58	management	married	tertiary	no	2143	yes	no
1	36	technician	single	secondary	no	265	yes	yes
2	25	blue-collar	married	secondary	no	-7	yes	no
3	53	technician	married	secondary	no	-3	no	no
4	24	technician	single	secondary	no	-103	yes	yes

	age	job	marital	education	default	balance	housing	loan
0	58	management	married	tertiary	0	2143	1	0
1	36	technician	single	secondary	0	265	1	1
2	25	blue-collar	married	secondary	0	-7	1	0
3	53	technician	married	secondary	0	-3	0	0
4	24	technician	single	secondary	0	-103	1	1

Label Encoding 실습

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

4) Scaling

① 개념: 특성(Feature)들의 값의 분포의 특성(평균, 표준편차) 또는 범위를 [0, 1]로 동일하게 해주는 작업

② 필요성: 특성마다 다른 분포나 범위를 가지는 경우 머신 러닝 모델들이 제대로 학습되지 않을 가능성이 있음

③ 종류

- 표준화(Standardization) : 데이터를 평균이 0이고, 표준 편차가 1인 분포로 변환

$$\frac{x-\mu}{\sigma} \quad (\mu : \text{평균}, \sigma : \text{표준편차})$$

- 정규화(Normalization) : 데이터를 범위가 0 ~ 1인 데이터로 변환

$$\frac{x-x_{min}}{x_{max}-x_{min}}$$

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

4) Scaling

④ 머신러닝 알고리즘별 scaling 가이드

알고리즘 분류	대표 모델	추천 전처리 방법	핵심 이유
선형 & 규제 모델	Linear / Logistic Regression, Ridge, Lasso	**StandardScaler**	L1/L2 규제 패널티 계산 시 필요
SVM & 차원 축소	SVM, PCA, LDA	**StandardScaler**	
딥러닝 (신경망)	MLP, CNN, RNN 등	**StandardScaler** 또는, (이상치 clip 후) **MinMaxScaler**	스케일러 1개만 적용
거리 기반 모델	K-Means	**이상치 clip 후, StandardScaler** 또는 **RobustScaler**	K-Means는 이상치에 극도로 취약
트리 기반 모델	Decision Tree, Random Forest, XGBoost, LightGBM	**스케일링 불필요 (Skip)**	스케일 변환이 성능에 전혀 영향 없음

Scaling 실습

데이터 전처리(preprocessing)

- Feature Engineering(특성 공학)

5) 구간화(Binning)

① 개념 : 연속형 수치 데이터를 특정 구간으로 나누어 범주형 데이터로 변환하는 작업

② 구현 방법

- pd.cut(x, bins, labels)
  - x : data
  - bins : 구간의 크기
  - labels:범주의 라벨

구간화(Binning) 실습

데이터 전처리(preprocessing)

- pd.DataFrame.query() 함수

1) 개념

① 데이터프레임에서 특정 조건을 만족하는 행(row)들만 추출(필터링)하는 함수

② 조건문을 문자열(string) 형태로 작성하기 때문에, 여러 조건을 조합할 때 코드가 복잡해지지 않고 직관적으로 사용할 수 있음

2) 구현 방법

① df.query('조건문 문자열') 형태로 사용

② 조건문 안에는 컬럼명과 비교/논리 연산자를 자유롭게 쓸 수 있음

③ 비교 연산자

- 두 변수의 크기를 비교할 수 있는 도구
- 결과 값 : 참(True), 거짓(False)

종류	설명
>	왼쪽 값이 크다
<	왼쪽 값이 작다
>=	왼쪽 값이 크거나 같다
<=	왼쪽 값이 작거나 같다
==	값이 같다
!=	값이 같지 않다

데이터 전처리(preprocessing)

- pd.DataFrame.query() 함수

2) 구현 방법

④ 논리 연산자

- 두 가지 이상의 조건(비교 연산)을 동시에 처리할 때 사용하는 도구
- 종류 : 논리곱 , 논리합, ~논리부정

논리 연산자	문법	설명
and	a and b	AND(논리곱), 양쪽 모두 참일 때 참
or	a or b	OR((논리합), 양쪽 중 한쪽만 참이라도 참
not	not x	NOT(논리부정), 참과 거짓을 뒤집음

pd.DataFrame.query() 함수 실습

데이터 전처리(preprocessing)

- pd.concat() 함수: 데이터프레임들을 병합한다.

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']},
                   index=[0, 1, 2, 3])

df2 = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']},
                   index=[4, 5, 6, 7])

df3 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],
                    'B': ['B8', 'B9', 'B10', 'B11'],
                    'C': ['C8', 'C9', 'C10', 'C11'],
                    'D': ['D8', 'D9', 'D10', 'D11']},
                   index=[8, 9, 10, 11])
```



df1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

df2

	A	B	C	D
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7

df3

	A	B	C	D
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11



1개의 DataFrame

데이터 전처리(preprocessing)

- pd.concat() 함수 사용법

```
# axis=0 방향으로 병합(기본값, default)
df_concat = pd.concat([df1,df2,df3], ignore_index=True)
```

axis=0



"공통된 컬럼을 기준으로,

y축 방향으로 병합"



	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

pd.concat() 함수 실습

10. 머신 러닝

- 지도 학습을 이용한 회귀(1)

회귀(Regression)

- 학습 데이터를 이용하여 데이터의 특성과 상관 관계 등을 파악하고, 그 결과를 바탕으로 학습 데이터에 없는 미지의 데이터가 주어졌을 경우에, 연속적인 숫자 값으로 예측하는 것
- 예 : 공부 시간과 시험 성적의 관계

x(hour)	y(score)
9	90
8	80
4	40
2	20

시험성적 데이터

7시간 공부 할 경우
성적은 몇 점 일까?

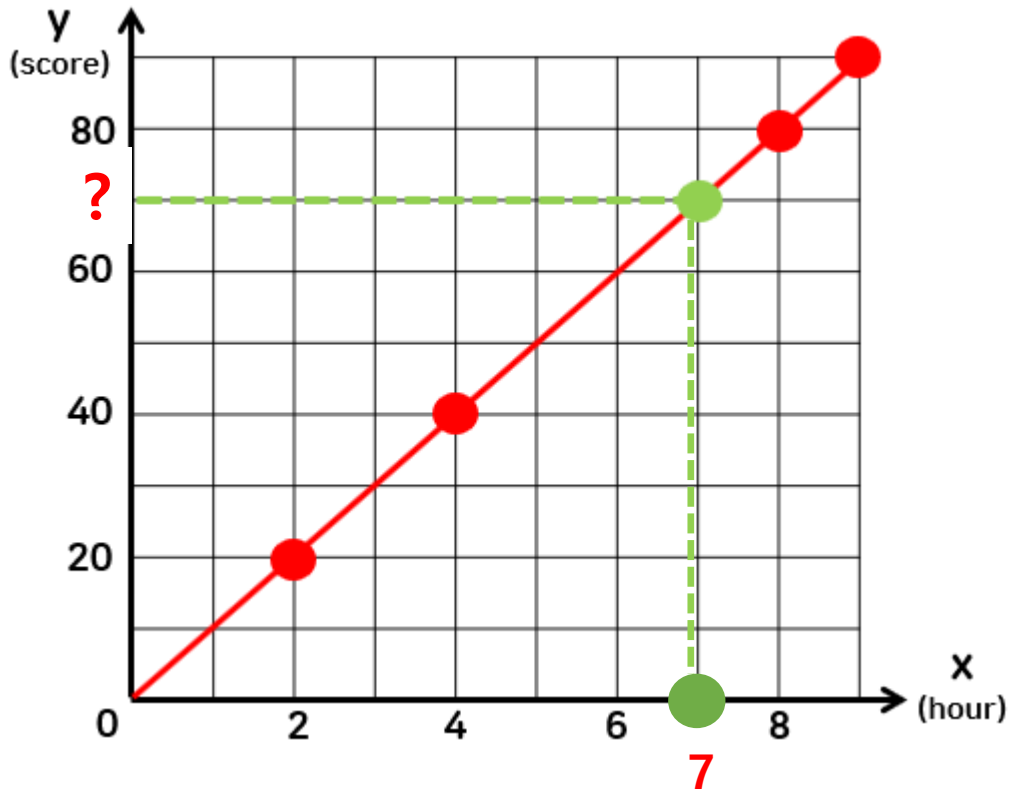
선형 회귀(Linear Regression)

- 학습 데이터의 입력과 출력의 관계를 직선의 형태로 나타낼 수 있고, 미지의 데이터도 y = Wx + b의 값으로 예측할 수 있는 것

x(hour)	y(score)
9	90
8	80
4	40
2	20

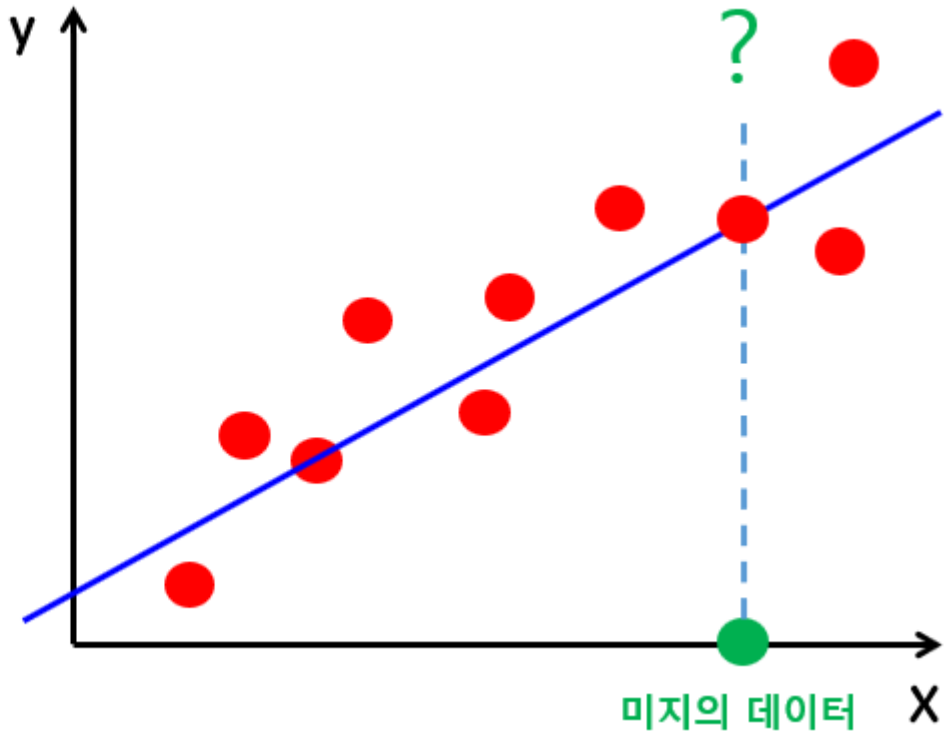
시험성적 데이터

7시간 공부 시 성적 ?



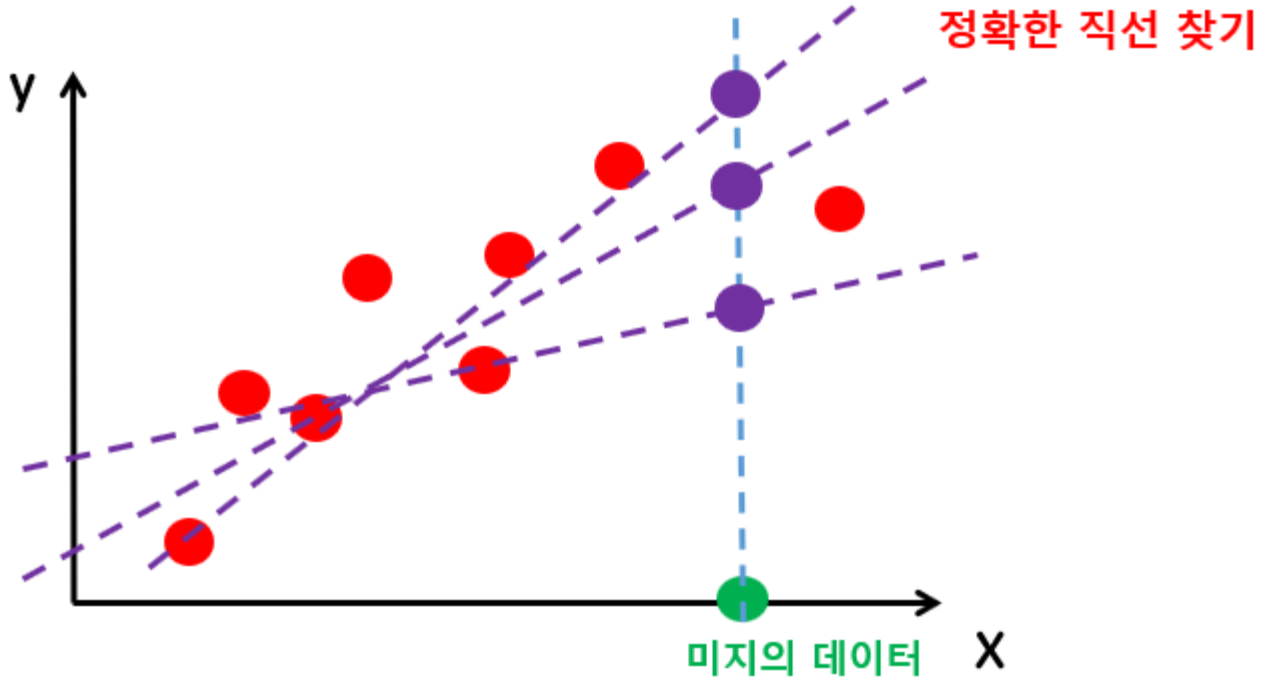
선형 회귀(Linear Regression)

X_train	y_train
공부시간(x)	시험성적(t)
9	74
14	81
21	86
27	88
32	90
37	92



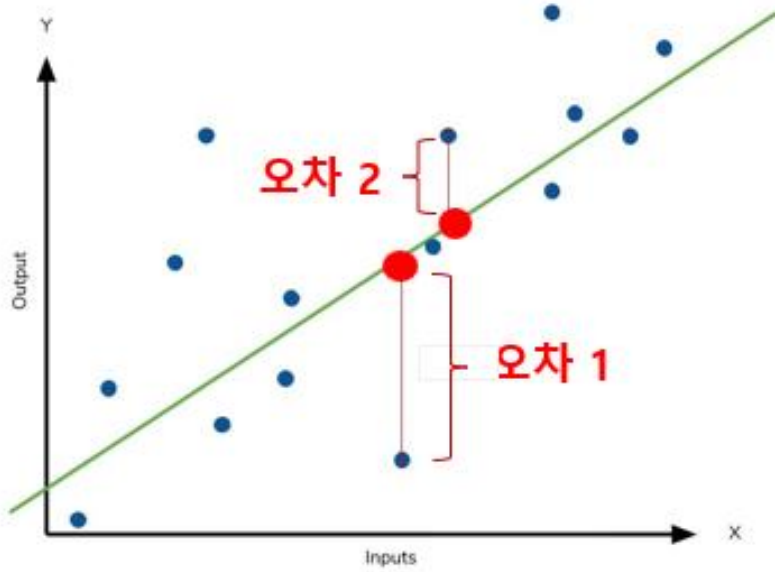
선형 회귀(Linear Regression) 정리

- 학습 데이터에는 없는 미지의 데이터에 대한 값을 예측할 때, 데이터의 분포를 가장 잘 표현할 수 있는 직선을 그려서 값을 예측하는 방법



선형 회귀 모델의 성능 평가 방법

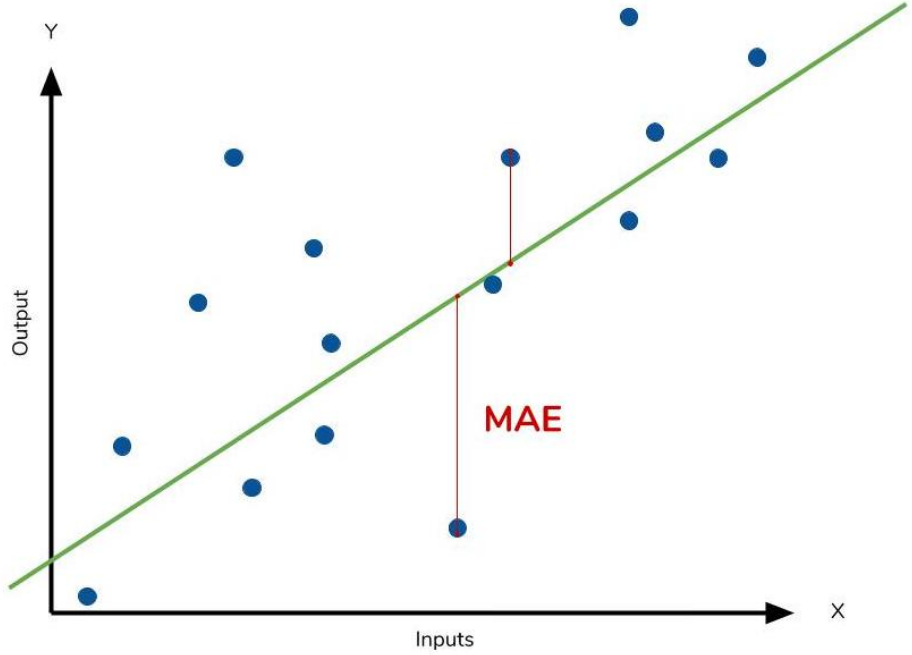
- 오차 또는 에러(Error) 또는 손실(loss)이 작을 수록 성능이 좋은 모델이다.



선형 회귀 모델의 성능 평가 방법

(1) 평균 절대값 오차(MAE : Mean Absolute Error)

- 개념 : 모델의 예측 값과 실제 값의 차이의 절대 값을 모두 더하여 평균을 구한다

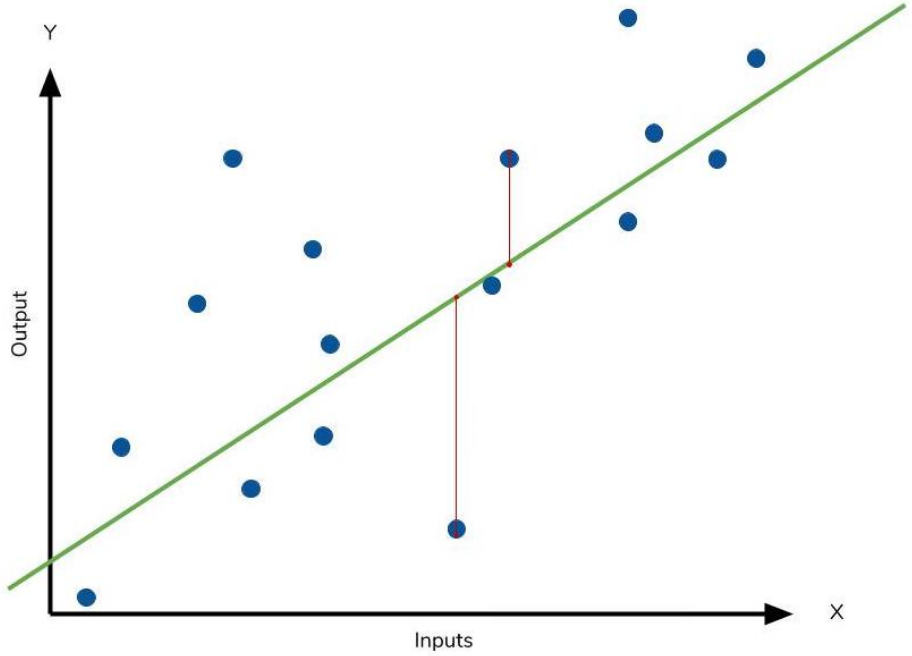


$$MAE = \frac{\sum |y - \hat{y}|}{n}$$

선형 회귀 모델의 성능 평가 방법

(2) 평균 제곱 오차(MSE : Mean Squared Error)

- 개념 : 모델의 예측 값과 실제 값의 차이의 제곱을 모두 더하여 평균을 구한다



$$MSE = \frac{\sum (y - \hat{y})^2}{n}$$

선형 회귀 모델의 성능 평가 방법

(3) 평균 제곱근 오차(RMSE : Root Mean Squared Error)

- 개념 : 평균 제곱 오차(MSE)에 Root를 씌운 것

$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$

선형 회귀 모델 - LinearRegression

- 평균 제곱근 오차(RMSE)가 최소가 되는 직선을 구하여 값을 예측하는 선형 회귀 모델
- 사용법

```
# 필요한 라이브러리 임폴트
from sklearn.linear_model import LinearRegression

# 선형 회귀 모델 생성 함수 호출, 회귀 모델 생성
lireg = LinearRegression()

# 선형 회귀 모델 학습, 예측
lireg.fit(X_train, y_train)

# 선형 회귀 모델 학습, 예측
y_pred = lireg.predict(X_test)
```

선형 회귀 모델 – LinearRegression 실습

- 보스턴 주택 가격 예측

1) 데이터 : 집 값에 영향을 주는 특성 13가지, 506개

2) 컬럼 정보

0	CRIM: 인구 1인당 범죄 발생 수
1	ZN: 25,000평방 피트 이상의 주거 구역 비중
2	INDUS: 소매업 외 상업이 차지하는 면적 비율
3	CHAS: 찰스강 위치 변수(1: 강 주변, 0: 이외)
4	NOX: 일산화질소 농도
5	RM: 집의 평균 방 수
6	AGE: 1940년 이전에 지어진 비율
7	DIS: 5가지 보스턴 시 고용 사설까지의 거리
8	RAD: 순환고속도로의 접근 용이성
9	TAX: $10,000당 부동산 세율 총계
10	PTRATIO: 지역별 학생과 교사 비율
11	B: 지역별 흑인 비율
12	LSTAT: 급여가 낮은 직업에 종사하는 인구 비율(%)

3) 목표: 집 값을 예측할 수 있는 LinearRegression 모델 학습 및 예측, 평가 진행

선형 회귀 모델 – LinearRegression 실습

- 보스턴 주택 가격 분석 및 예측

다중 선형 회귀

- 데이터의 특성(feature)을 여러 개 사용하는 선형 회귀 사용법

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_p x_p + b$$

$$y = wx + b,$$

단순 선형 회귀에 비해서 학습에 적용하는

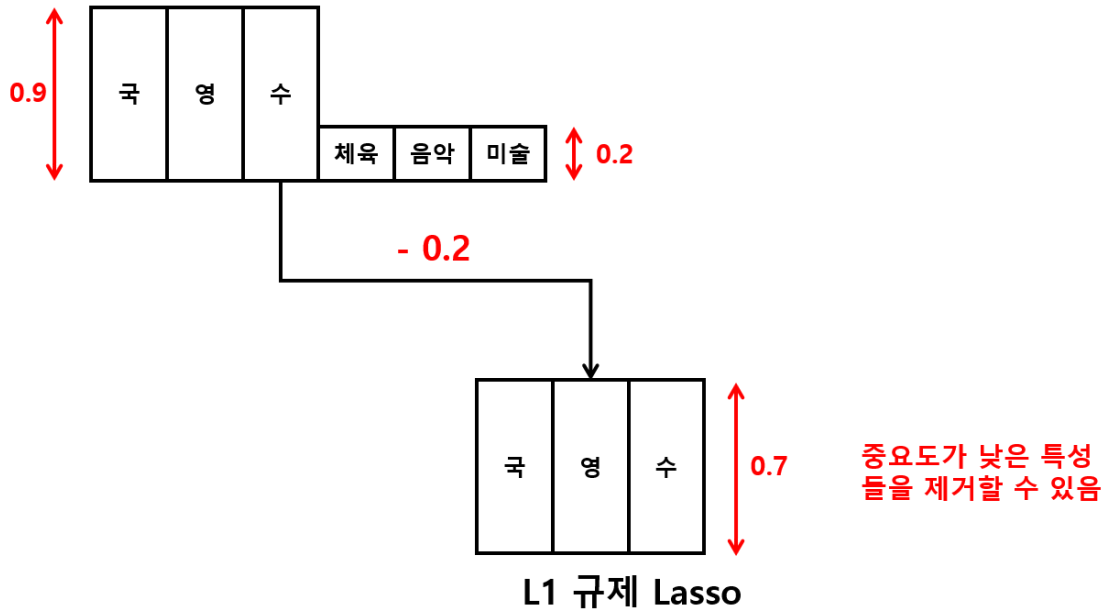
특성이 많고 복잡하다

모델 정규화(Regularization)

- 모델이 과대적합 되지 않도록 복잡한 모델을 단순하게 만드는 것
- 데이터의 특성(feature)에는 손대지 않으면서 복잡도를 줄이려면, **가중치(W)를 0으로 만들거나 0에 가까운 작은 숫자로 만들면 된다.**

모델 정규화(Regularization) 방법 1 - L1 규제

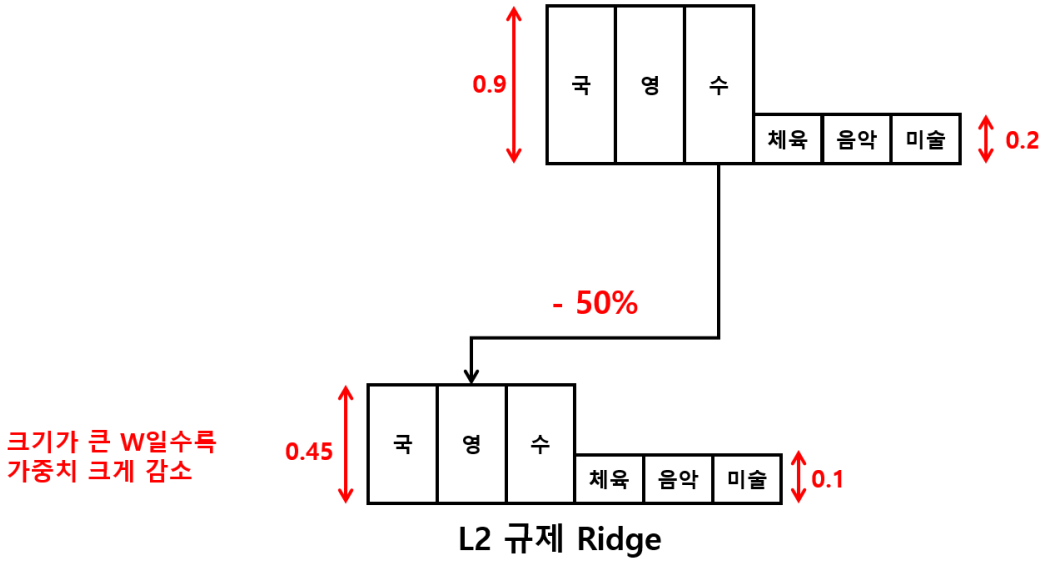
- 모든 가중치 W를 같은 크기로 감소 : $w1$ - k, $w2$ - k, ..., wn - k,
- 효과 : 크기가 작은 가중치들을 0으로 만들어서 모델을 단순하게 만든다
- L1 규제 적용 선형 회귀 모델 : Lasso 모델



하이퍼파라미터 α → 규제의 강도 조절

모델 정규화(Regularization) 방법 2 – L2 규제

- 모든 가중치 W를 같은 비율로 감소 : $w1$ = 0.9, $w2$ = 0.2 → $w1$ = 0.45, $w2$ = 0.1, …
- 효과 : 크기가 큰 가중치일수록 더 크게 감소하게 만들어서 전체 가중치의 크기를 0에 가깝게 만들어서 처음보다 모델을 단순하게 만든다
- L2 규제 적용 선형 회귀 모델 : Ridge 모델



하이퍼파라미터 α → 규제의 강도 조절

Lasso 모델

▶ 사용법

```
# 필요한 라이브러리 임폴트
from sklearn.linear_model import Lasso

# Lasso 모델 생성 함수 호출, 하이퍼파라미터 설정
lasso = Lasso (random_state)

# Lasso 모델 학습, 예측
lasso.fit(X_train, y_train)

# Lasso 모델 예측
lasso.predict(X_test)
```

Ridge 모델

▶ 사용법

```
# 필요한 라이브러리 임폴트
from sklearn.linear_model import Ridge

# Ridge 모델 생성 함수 호출, 하이퍼파라미터 설정
ridge = Ridge (random_state)

# Ridge 모델 학습, 예측
ridge.fit(X_train, y_train)

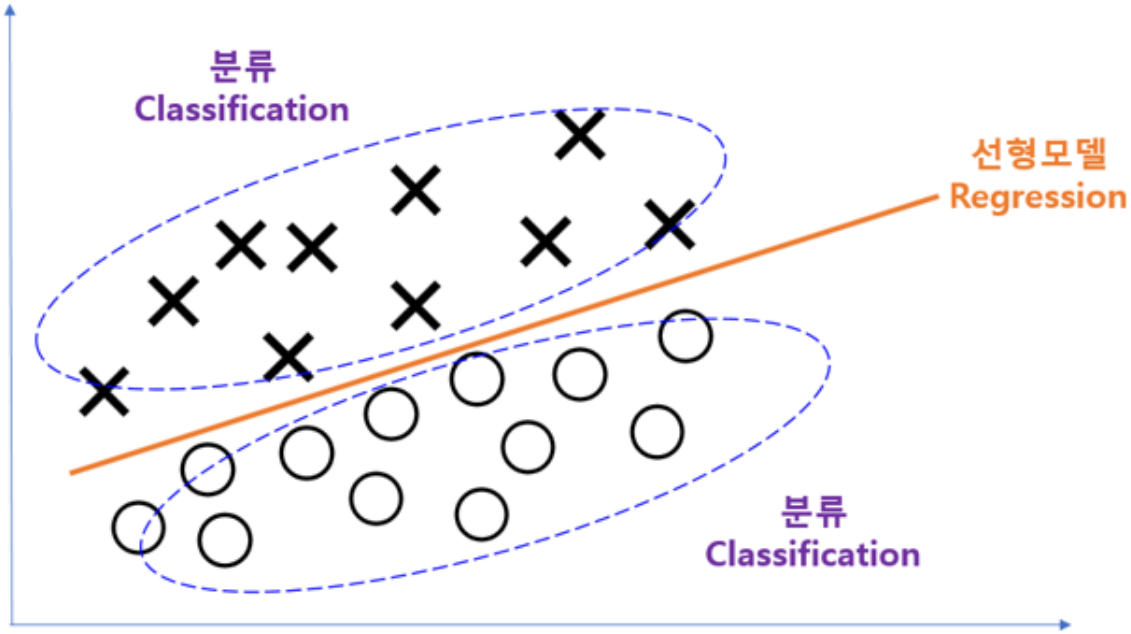
# Ridge 모델 예측
ridge.predict(X_test)
```

11. 머신 러닝

- 지도 학습을 이용한 분류

주요 분류 모델(알고리즘): 로지스틱 회귀

- 선형 회귀 모델을 이용한 분류 알고리즘: "이름은 회귀, 목적은 분류"



주요 분류 모델(알고리즘): 로지스틱 회귀

- 핵심 원리

1) 선형 회귀의 결과값(양수/음수)으로 분류하지 않고, 시그모이드(Sigmoid) 함수를 거쳐 0과 1 사이의 확률로 변환

① 단순히 양수/음수로만 판단하면 "A와 B 중에서 A 클래스다 " 라는 결과만 알 수 있지만, 시그모이드 함수를 사용하면 "A클래스일 확률이 95%다"라는 정보를 얻을 수 있음

② 예시: 암 진단에서 결과값이 0.51(확률 51%)인 경우와 0.99(확률 99%)인 경우, 의사는 서로 다른 대응 전략을 세울 수 있음

2) 시그모이드(Sigmoid) 함수

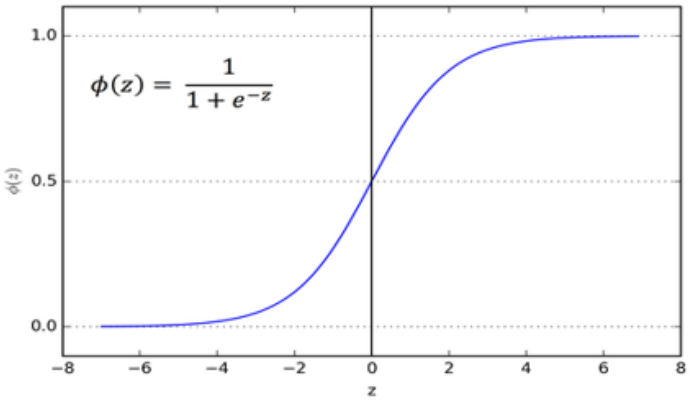
① - ∞ ~ + ∞ 사이의 입력 값을 0에서 1 사이의 값으로 압축하는 S자형 곡선 함수

② 시그모이드 함수 값 = A 클래스일 확률 P → B 클래스 일 확률 = 1 – P

③ 결정 경계: 시그모이드 함수 값이 0.5가 되는 지점

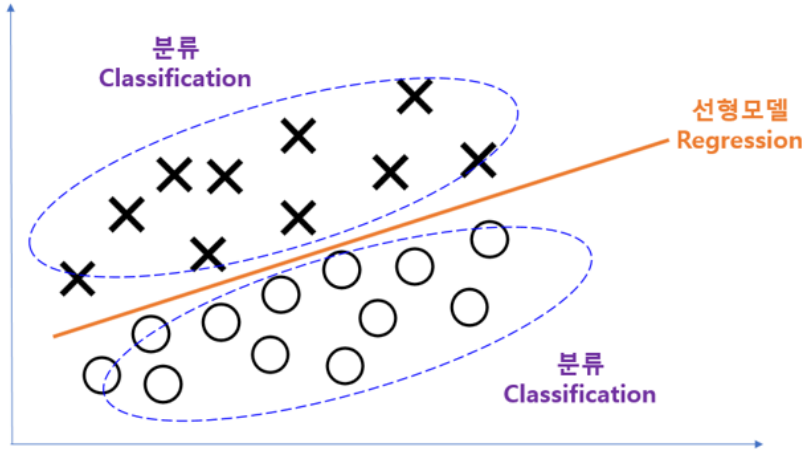
④ 시그모이드 함수 값

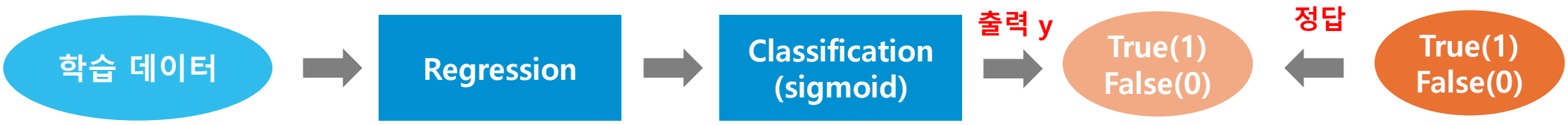
- 입력 값: - ∞ → 시그모이드 함수 값(확률 P) = 0
- 입력 값: 0 → 시그모이드 함수 값(확률 P) = 0.5
- 입력 값: + ∞ → 시그모이드 함수 값(확률 P) = 1



주요 분류 모델(알고리즘): 로지스틱 회귀

- 정리





$$y = \text{sigmoid}(z) = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

주요 분류 모델(알고리즘): 로지스틱 회귀

- 모델(sklearn.linear_model.LogisticRegression)의 학습 과정: 손실(loss) 최적화

1) 개념: 정답이 1일 때 예측 확률을 1에 가깝게, 정답이 0일 때 예측 확률을 0에 가깝게 만드는 최적의 선형 회귀 직선을 찾겠다는 뜻 → w1·x1 + w2·x2 + ... + b

2) 모델의 매개 변수 max_iter, 속성 변수 coef_ 의미

① max_iter: 학습(선형 회귀 직선의 기울기 조정) 횟수

② coef_: 최적의 선형 회귀 직선(w1·x1 + w2·x2 + ... + b)의 기울기 → w1, w2, ...

주요 분류 모델(알고리즘): 로지스틱 회귀

- 장점과 단점

1) 장점: 매우 빠른 학습 속도

2) 단점: 직선 형태의 경계만 가능하여 비선형 데이터에 한계

주요 분류 모델(알고리즘): 로지스틱 회귀 실습

- 타이타닉 생존 여부 예측

1) 1,309명의 데이터 → 학습용 데이터(train) 891명 + 평가용 데이터(test) 418명

① 학습용 데이터의 모양(shape) = (891, 12)

② 평가용 데이터의 모양(shape) = (418, 12)

2) 컬럼 정보

Survived : 생존 여부
Pclass : Passenger Class, 승객 등급
Name : 승객 이름
Sex : 승객 성별
Age : 승객 나이
Sibsp : 탑승 한 형제/배우자 수
Parch : 탑승 한 부모/자녀 수
Ticket : 티켓 번호
Fare : 승객 지불 요금
Cabin : 선실 이름
Embarked : 승선항 (C = 쉘 부르그, Q = 퀸즈타운, S = 사우스 햄튼)

3) 목표

① 데이터 전처리

② 학습용 데이터로 학습 후 평가용 데이터를 이용 생존 여부 분류 / 모델 평가

주요 분류 모델(알고리즘): 로지스틱 회귀 실습

- 타이타닉 생존 여부 예측

1) 데이터 불러오기

2) 데이터 전처리

① 데이터 병합: pd.concat([df1, df2], ignore_index=True)

② 컬럼별 누락 데이터 처리

3) 불필요한 컬럼 삭제

4) One-Hot encoding

5) 결과 저장하기

6) 학습용 데이터와 평가용 데이터 생성

7) 표준화

주요 분류 모델(알고리즘): 로지스틱 회귀 실습

- 타이타닉 생존 여부 예측

8) LogisticRegression 모델

① 모델 생성

```
# 필요한 라이브러리 임폴트
from sklearn.linear_model import LogisticRegression
# LogisticRegression 모델 생성 함수 호출, 모델 생성
logistic = LogisticRegression()
```

② 모델 학습

```
logistic.fit(X_train, y_train)
```

③ 평가용 데이터를 이용한 예측

```
pred_test = logistic.predict(X_test)
```

④ 모델 평가

```
# 필요한 함수 임폴트
from sklearn.metrics import accuracy_score
# 평가용 데이터에 대한 성능 평가
acc_lr = accuracy_score(y_test, pred_test)
```

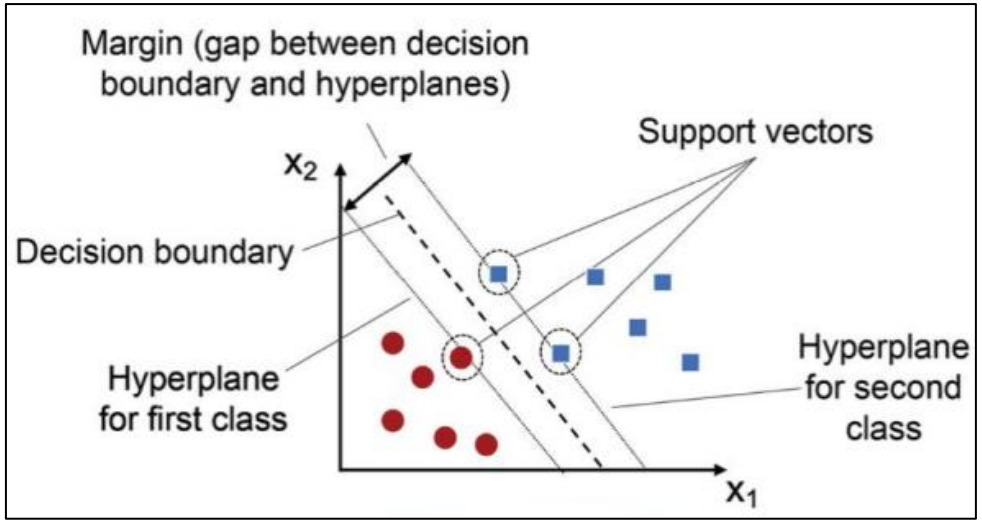
주요 분류 모델(알고리즘): 로지스틱 회귀 실습

- 타이타닉 생존 여부 예측 실습

주요 분류 모델(알고리즘): SVM

- 클래스 간의 안전한 거리(마진, Margin)을 최대화하여 모델의 성능을 높이는 분류 알고리즘

- 용어 정리
  1) 서포트 벡터(Support Vector): 결정 경계와 가장 가까운 위치의 데이터
  2) 마진(Margin) 최대화: 결정 경계와 서포트 벡터 사이의 거리



- 목표: 데이터를 두 클래스로 완벽하게 분리하면서도 마진이 최대화 될 수 있는 결정 경계 탐색

주요 분류 모델(알고리즘): SVM

- 장점

  1) 소규모 데이터에 강함: 적은 수의 데이터(수십~수백 개)에서도 일반화 능력이 탁월함

  2) 고차원 효율성: 피처(Feature) 수가 샘플 수보다 많을 때도 안정적인 성능 유지

- 단점

  • 연산 복잡도: 데이터가 커지면 학습 속도가 기하급수적으로 느려짐

SVM(support vector machine) 분류 모델 사용법

1. 필요한 라이브러리 임폴트

```
from sklearn.svm import SVC
```

2. 데이터 전처리

3. 학습용 데이터와 평가용 데이터 생성

4. 표준화

5. 모델 생성

```
# SVC 모델 생성 함수 호출, 모델 생성
   svc = SVC()
```

SVM(support vector machine) 분류 모델 사용법

6. 모델 학습

```
svc.fit(X_train_scaled, y_train)
```

7. 평가용 데이터를 이용한 예측

```
pred_test = svc.predict(X_test_scaled)
```

8. 모델 평가

```
accuracy = accuracy_score(y_test, pred_test)
```

SVM(support vector machine) 분류 모델 실습

- 타이타닉 생존 여부 예측

1) 1,309명의 데이터 → 학습용 데이터(train) 891명 + 평가용 데이터(test) 418명

① 학습용 데이터의 모양(shape) = (891, 12)

② 평가용 데이터의 모양(shape) = (418, 12)

2) 컬럼 정보

Survived : 생존 여부
Pclass : Passenger Class, 승객 등급
Name : 승객 이름
Sex : 승객 성별
Age : 승객 나이
Sibsp : 탑승 한 형제/배우자 수
Parch : 탑승 한 부모/자녀 수
Ticket : 티켓 번호
Fare : 승객 지불 요금
Cabin : 선실 이름
Embarked : 승선항 (C = 쉘 부르그, Q = 퀸즈타운, S = 사우스 햄튼)

3) 목표

① 데이터 전처리가 완료된 데이터를 이용

② 학습용 데이터로 학습 후 평가용 데이터를 이용 생존 여부 분류 / 모델 평가

SVM(support vector machine) 회귀 모델 사용법

1. 필요한 라이브러리 임폴트

```
from sklearn.svm import SVR
```

2. 데이터 전처리

3. 학습용 데이터와 평가용 데이터 생성

4. 표준화

5. 모델 생성

```
# SVR 모델 생성 함수 호출, 모델 생성
   svr = SVR()
```

SVM(support vector machine) 회귀 모델 사용법

6. 모델 학습

```
svr.fit(X_train_scaled, y_train)
```

7. 평가용 데이터를 이용한 예측

```
pred_test = svr.predict(X_test_scaled)
```

8. 모델 평가

```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import root_mean_squared_error as rmse
rmse = rmse(y_test, pred_test)
```

SVM(support vector machine) 회귀 모델 실습

- 보스턴 주택 가격 예측

1) 데이터 : 집 값에 영향을 주는 특성 13가지, 506개

2) 컬럼 정보

0	CRIM: 인구 1인당 범죄 발생 수
1	ZN: 25,000평방 피트 이상의 주거 구역 비중
2	INDUS: 소매업 외 상업이 차지하는 면적 비율
3	CHAS: 찰스강 위치 변수(1: 강 주변, 0: 이외)
4	NOX: 일산화질소 농도
5	RM: 집의 평균 방 수
6	AGE: 1940년 이전에 지어진 비율
7	DIS: 5가지 보스턴 시 고용 시설까지의 거리
8	RAD: 순환고속도로의 접근 용이성
9	TAX: $10,000당 부동산 세율 총계
10	PTRATIO: 지역별 학생과 교사 비율
11	B: 지역별 흑인 비율
12	LSTAT: 급여가 낮은 직업에 종사하는 인구 비율(%)

3) 목표

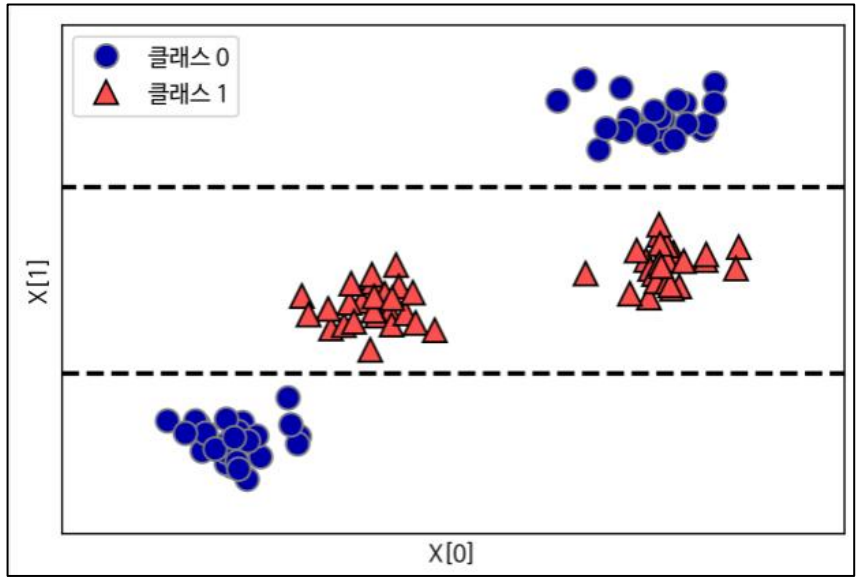
① 데이터 전처리

② 학습용 데이터로 학습 후 평가용 데이터를 이용 모델 성능 평가(rmse)

SVM(support vector machine) 회귀 모델 실습

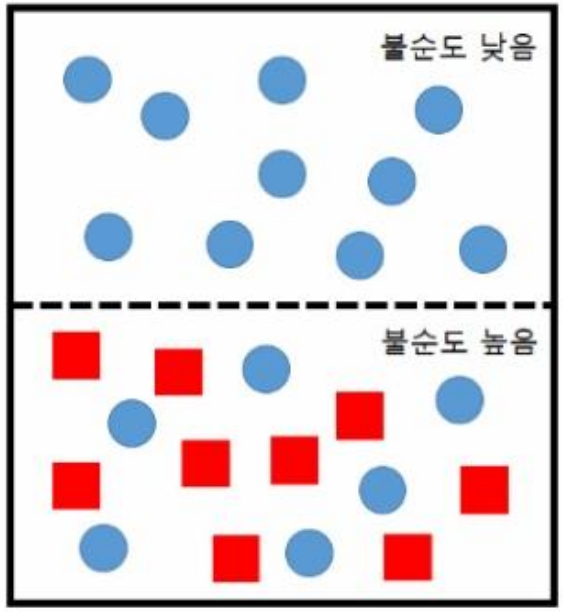
주요 분류 모델(알고리즘): 단일 결정 트리

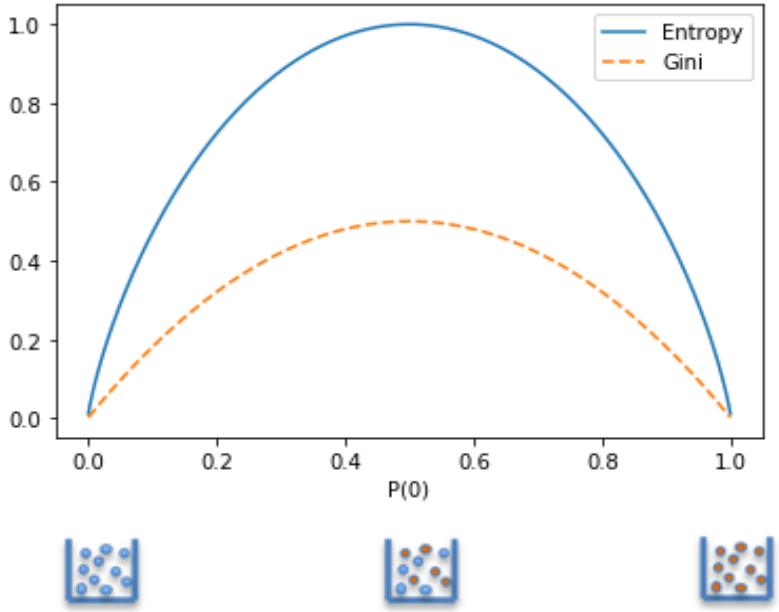
- 특정 특성의 임계값(Threshold)을 기준으로 데이터를 축에 수직인 경계로 반복 분할하여 영역 내의 불순도(Impurity)를 최소화하는 대표적인 비선형 알고리즘
- 용어 정리: 불순도(Impurity)
  1) 서로 다른 데이터가 섞여 있는 정도
     ① 최악의 불순도: 한 그룹에 여러 클래스가 정확히 반반씩 섞여 있어, 어느 쪽인지 도저히 예측할 수 없는 상태
     ② 불순도 0(순도 100%): 한 영역에 'A'라는 클래스만 완벽하게 모여 있는 상태
  2) 결정 트리의 목표는 뒤죽박죽 섞여 있는 데이터(불순도가 높은 상태)를 끼리끼리 모여 있는 상태(불순도가 낮은 상태)로 분리하는 것



용어 정리: 불순도

- 서로 다른 데이터가 섞여 있는 정도
  - 다양한 개체들이 섞여 있을수록 불순도가 높아짐
- 불순도 함수 - 불순도를 수치적으로 나타내는 함수
  - 지니 지수(Gini) / 엔트로피 지수(Entropy)





불순도 지표: 지니 계수(Gini) vs 엔트로피(Entropy)

- 지니 불순도 (Gini Impurity)

  1) 기본값(criterion='gini'). 계산 속도가 빠름

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

(단, p_{i,k}는 i번째 노드에 있는 훈련 샘플 중 클래스 k에 속한 샘플의 비율)

- 엔트로피 (Entropy / 정보 이득)

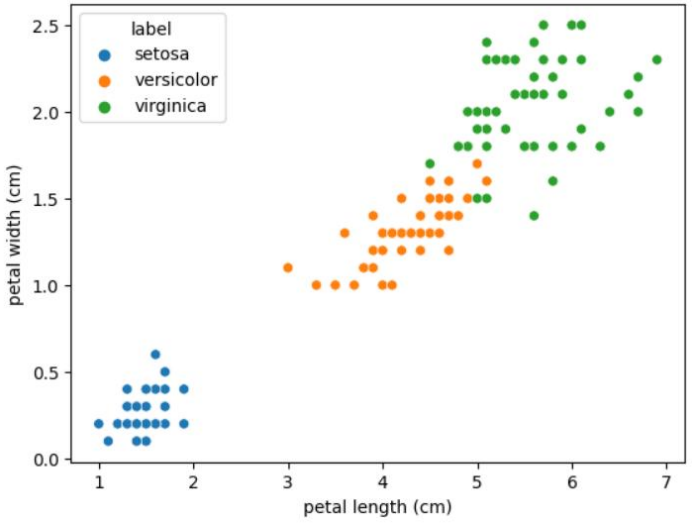
  : 열역학/정보이론의 혼잡도 개념(criterion='entropy')

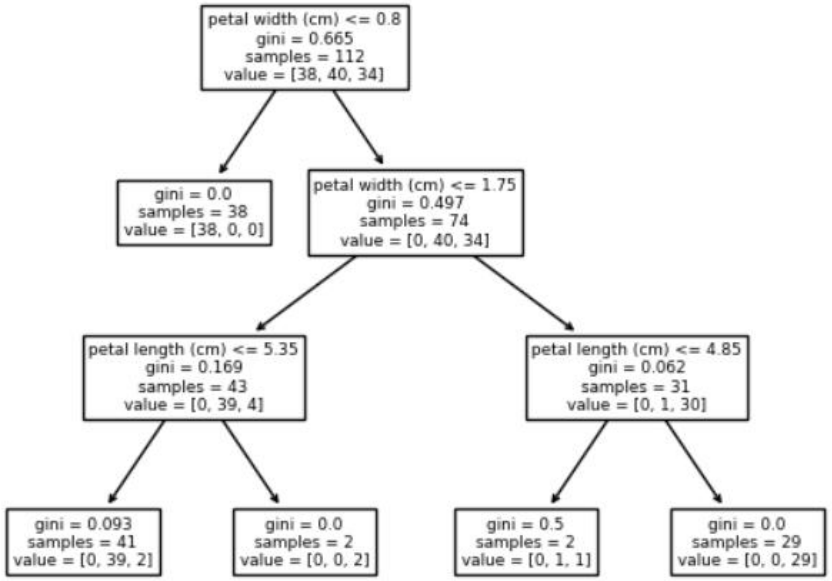
$$H_i = -\sum_{k=1, p_{i,k} \neq 0}^{n} p_{i,k} \log_2(p_{i,k})$$

주요 분류 모델(알고리즘): 단일 결정 트리

- 알고리즘에 대한 이해: "데이터 분할의 논리" → IF-THEN 규칙(AI가 하는 스무고개)

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
7.0	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6.0	2.5	virginica
5.8	2.7	5.1	1.9	virginica
7.1	3.0	5.9	2.1	virginica

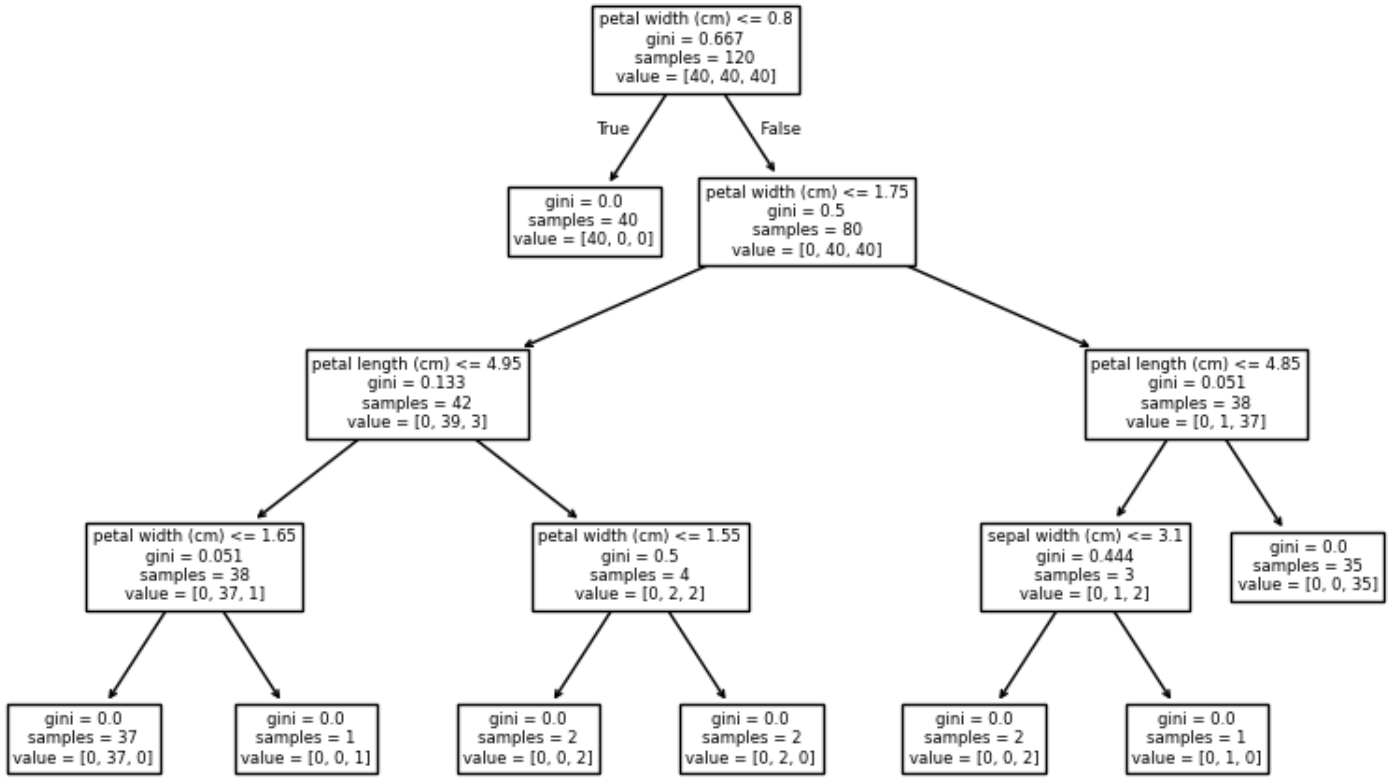




- 질문을 거듭할수록 데이터 그룹의 불순도가 낮아지며 한가지 품종으로 나뉜다.
- 해석 가능성 (Explainability): 모델의 예측 근거와 판단 과정을 시각적으로 확인 가능, "AI가 그냥 품종이 setosa라고 했어요"가 아니라, "꽃잎의 너비가 0.8cm 이하라서 품종이 setosa 입니다"라고 결과를 논리적으로 설명할 수 있다.

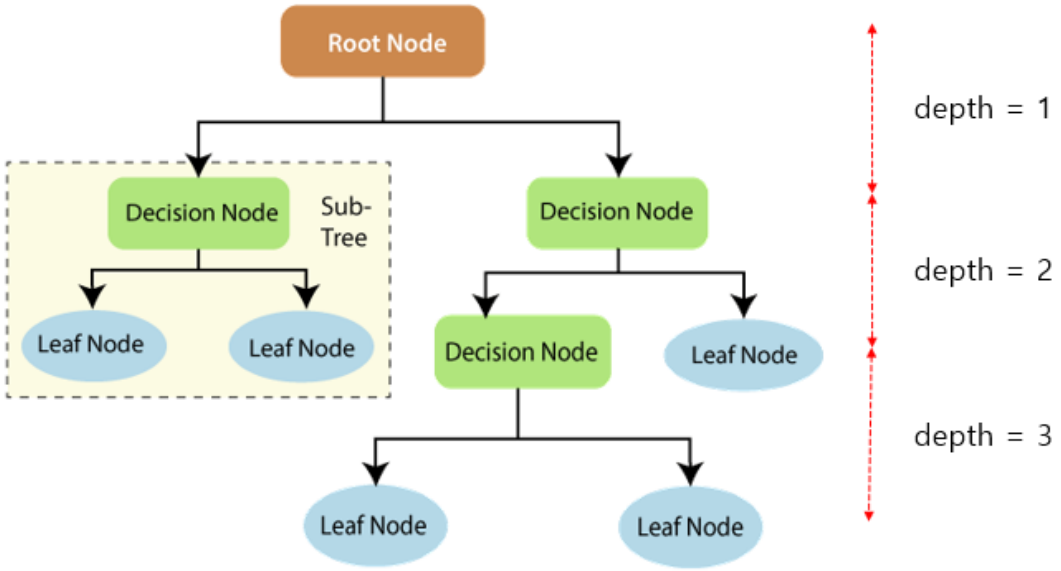
주요 분류 모델(알고리즘): 단일 결정 트리

- 이름이 결정 트리(Decision Tree)인 이유: 일련의 분류 규칙 적용(= 의사결정)을 통해 데이터를 순차적으로 분할하는 과정에 대한 시각적 표현



주요 분류 모델(알고리즘): 단일 결정 트리

- 트리의 주요 구성 요소



1) 루트 노드(Root Node): 최상단 분할 노드

2) 리프 노드(Leaf Node): 최종 클래스 또는 예측값이 결정되는 최하단 노드

3) 노드 정보

: Threshold (분할 조건), gini/entropy (불순도), samples (샘플 수), value (클래스별 샘플 수)

클래스 확률 추정 (Class Probability Estimation)

- 클래스 확률 추정의 원리

  : 결정 트리는 샘플이 어떤 리프 노드(Leaf Node)에 도달하면, 해당 리프 노드에 존재하는 학습용 데이터들의 클래스 비율을 기반으로 확률을 추정함

- 추정 프로세스

  1) 새로운 데이터 X가 트리를 따라 최종 리프 노드 L에 도착

  2) 해당 리프 노드 L 안에 속한 클래스 k의 샘플 수 비율을 계산

$$P(Y = k \mid X) = \frac{N_{L,k}}{N_L}$$

  3) 가장 높은 확률을 가진 클래스를 최종 예측값으로 반환

  4) model.predict(X): 확률이 가장 높은 클래스를 출력

CART 알고리즘 (Classification and Regression Trees)

- CART 알고리즘의 특징

  1) Scikit-Learn이 결정 트리를 훈련할 때 사용하는 대표적인 알고리즘

  2) 이진 트리(Binary Tree)만을 생성 (리프 노드를 제외한 모든 노드는 항상 2개의 자식 노드로 분할됨)

- 분할 기준

  • 하나의 피처 k와 임계값 t_k를 사용하여 학습용 데이터를 가장 순수한(불순도가 낮은) 두 개의 서브셋으로 나눔

- 정지(Stopping) 기준

  • 사전에 설정한 최대 깊이(max_depth)에 도달 또는 더 이상 불순도를 줄일 수 있는 분할이 없을 때 분할 중단

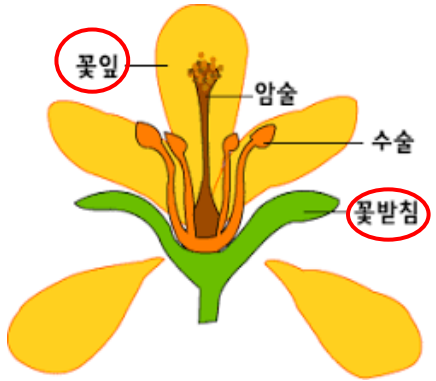
트리 기반 분류 모델 실습

- 데이터 : iris 데이터

1) 150개의 붓꽃 데이터

2) 컬럼 정보 : 4개의 특성 + 품종 레이블

- sepal length (cm) : 꽃받침의 길이
- sepal width (cm) : 꽃받침의 너비
- petal length (cm) : 꽃잎의 길이
- petal width (cm) : 꽃잎의 너비
- label : 품종 레이블





3) 실습 목적 : 붓꽃의 품종을 분류하는 트리기반 머신 러닝 모델의 원리 이해

트리 기반 분류 모델 실습

- 데이터 : Diabetes Health Indicators Dataset(당뇨병 건강 지표 데이터)
  1) 미국 질병통제예방센터(CDC)에서 매년 수집하는 건강 관련 전화 설문조사 데이터
  2) 70692개의 데이터(컬럼 22개), 당뇨병 분류 결과 제공
  3) 실습 목적 : 컬럼 21개 → 탐색적 데이터 분석, 모델 학습 → 당뇨병 여부 분류(예측)

#	Column	Non-Null Count	Dtype
0	Diabetes_binary	70692 non-null	int64
1	HighBP	70692 non-null	int64
2	HighChol	70692 non-null	int64
3	CholCheck	70692 non-null	int64
4	BMI	70692 non-null	int64
5	Smoker	70692 non-null	int64
6	Stroke	70692 non-null	int64
7	HeartDiseaseorAttack	70692 non-null	int64
8	PhysActivity	70692 non-null	int64
9	Fruits	70692 non-null	int64
10	Veggies	70692 non-null	int64
11	HvyAlcoholConsump	70692 non-null	int64
12	AnyHealthcare	70692 non-null	int64
13	NoDocbcCost	70692 non-null	int64
14	GenHlth	70692 non-null	int64
15	MentHlth	70692 non-null	int64
16	PhysHlth	70692 non-null	int64
17	DiffWalk	70692 non-null	int64
18	Gender	70692 non-null	int64
19	Age	70692 non-null	int64
20	Education	70692 non-null	int64
21	Income	70692 non-null	int64

트리 기반 분류 모델 실습

- 컬럼 정보

1) Diabetes_binary : 정답 레이블, 0 = 정상, 1 = 당뇨병 + 당뇨병 전단계
2) HighBP : 고혈합 유무, 0 = 정상, 1 = 고혈압
3) HighChol : 고지혈 유무, 0 = 정상, 1 = 고지혈
4) CholCheck : 5년 동안에 고지혈 검사 여부, 0 = no, 1 = yes
5) BMI : 체질량지수, 몸무게(kg)를 키의 제곱(㎡)으로 나눈 값
6) Smoker : 최소 100개비 이상 흡연 여부, 0 = no, 1 = yes
7) Stroke : 뇌졸증 여부, 0 = no, 1 = yes
8) HeartDiseaseorAttack : 심근 경색과 같은 심장 질환 여부, 0 = no, 1 = yes
9) PhysActivity : 지난 30일 동안 운동 여부, 0 = no, 1 = yes
10) Fruits : 1일 1회 이상 과일 섭취 여부, 0 = no, 1 = yes

트리 기반 분류 모델 실습

- 컬럼 정보

11) Veggies : 1일 1회 이상 야채 섭취 여부, 0 = no, 1 = yes

12) HvyAlcoholConsump : 음주 여부, 0 = no, 1 = yes

```
### 음주의 기준
# adult men >=14 drinks per week
# adult women>=7 drinks per week
```

13) AnyHealthcare : 건강 보장 보험 보유 여부, 0 = no, 1 = yes

14) NoDocbcCost : 0 = no, 1 = yes

15) GenHlth : 본인이 생각하는 자신의 전반적인 건강 상태 평가, 1 – 5단계

1 = excellent, 2 = very good, 3 = good, 4 = fair, 5 = poor

16) MentHlth : 본인이 생각하는 자신의 정신 건강 상태 평가

질문 : 스트레스, 우울증을 포함하여 지난 30일 중 며칠 동안 정신 건강이 좋지 않았었나요?

답변 : 정신 건강이 좋지 않았던 일 수

트리 기반 분류 모델 실습

- 컬럼 정보

17) PhysHlth : 본인이 생각하는 자신의 신체 건강 상태 평가

질문 : 신체적 질병과 부상을 포함하여 포함하여 지난 30일 중 며칠 동안 신체적 건강이 좋지 않았었나요?

답변 : 정신 건강이 좋지 않았던 일 수

18) DiffWalk : 0 = no, 1 = yes

질문 : 당신은 걷거나 계단을 오르는 데 심각한 어려움이 있습니까?

답변 : 아니오(0), 예(1)

19) Gender : 응답자의 성별, 0 = female, 1 = male

20) Age : 응답자의 나이, 1 – 13 단계로 범주화

1단계 : 18 – 24 / 2단계 : 25 – 29 / 3단계 : 30 – 34 / 4단계 : 35 – 39 / 5단계 : 40 - 44

6단계 : 45 – 49 / 7단계 : 50 – 54 / 8단계 : 55 – 59 / 9단계 : 60 – 64 / 10단계 : 65 - 69

11단계 : 70 – 74 / 12단계 : 75 – 79 / 13단계 : 80 or older

트리 기반 분류 모델 실습

- 컬럼 정보

21) Education : 응답자의 교육 수준. 1 - 6단계 범주화

1단계 : Never attended school / 2단계 Elementary / 3단계 Some high school

4단계 : High school graduate / 5단계 : Some college or technical school / 6단계 : College graduate

22) Income : 응답자의 연간 가구 소득 수준, 1 - 8단계 범주화

1단계 : Less than $10,000 / 2단계 : Less than $15,000 / 3단계 : Less than $20,000

4단계 : Less than $25,000 / 5단계 : Less than $35,000 / 6단계 : Less than $50,000

7단계 : Less than $75,000 / 8단계 : $75,000 or more

트리 기반 분류 모델 실습

- 데이터 불러오기
- 학습용과 평가용 데이터 생성
- 결정 트리 분류 모델 생성 함수 호출, 분류 모델 생성

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(매개변수)
```

- 모델 학습

```
dt.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test = dt.predict(X_test)
```

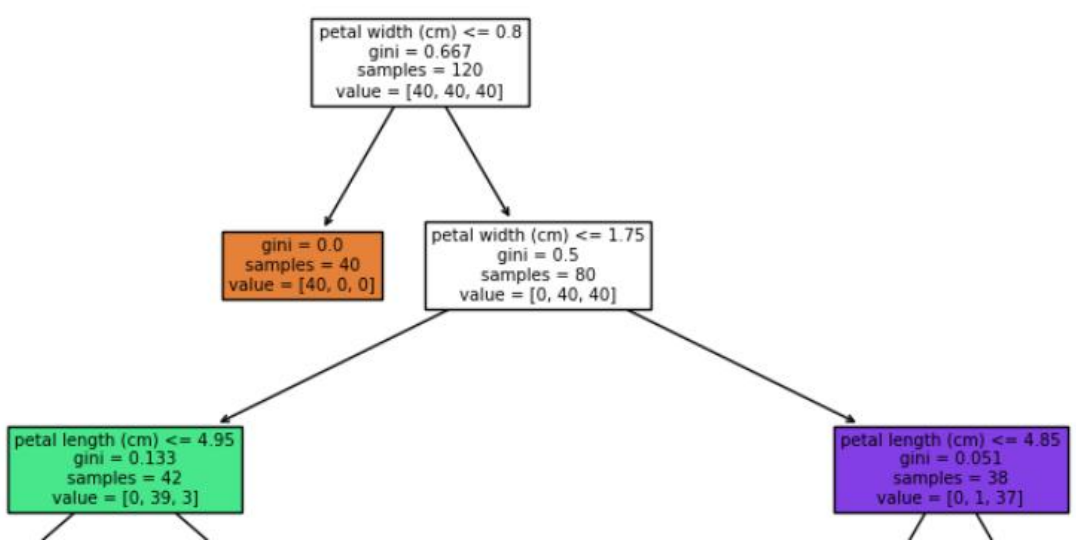
- 모델 평가

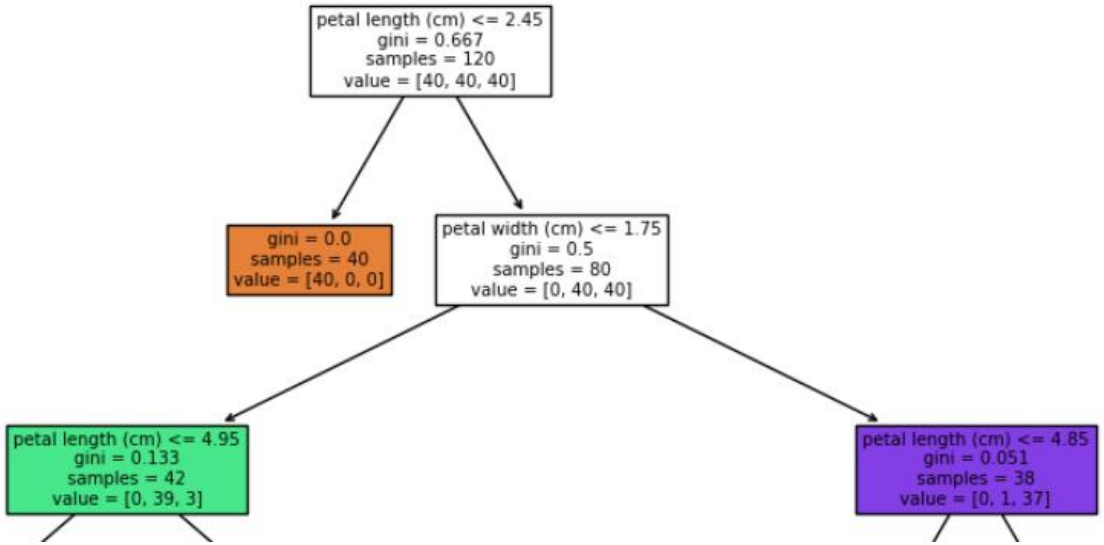
```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import accuracy_score
# 평가용 데이터에 대한 성능 측정
acc_dt = accuracy_score (y_test, pred_test)
```

트리 기반 분류 모델 실습

- 주요 매개변수: random_state (재현성)

1) 결정 트리는 각 노드에서 불순도가 최소화되는 방향으로 분할 기준을 결정.
2) 같은 불순도 감소량을 가진 여러 가지 분할 기준이 존재할 수 있음
3) random_state는 이러한 여러 분할 기준 중 하나를 선택하는 과정에 영향을 줄 수 있음
4) 선택된 분할 기준에 따라 다음 단계에서 분할되는 방향이 달라지고, 이는 트리 구조와 모델 성능에 영향을 줄 수 있음





트리 기반 분류 모델 실습

주요 분류 모델(알고리즘): 단일 결정 트리

- 장점
  1) 최고의 설명력: 판단 근거를 시각적으로 보여주는 '화이트박스' 모델
  2) 비선형성: 복잡한 비선형 관계도 직관적으로 학습 가능
- 단점
  1) 과대적합(Overfitting): 데이터의 아주 미세한 노이즈까지 학습하려는 성향이 강함, 트리가 깊어질(Deep)수록 훈련 데이터는 100% 완벽하게 맞추지만, 조금만 새로운 데이터가 들어와도 예측이 완전히 틀려 성능이 급격히 떨어짐
  2) 성능 한계: 단일 모델로는 높은 정확도를 얻기 어려움

과대적합 방지를 위한 규제 매개변수 (Regularization)

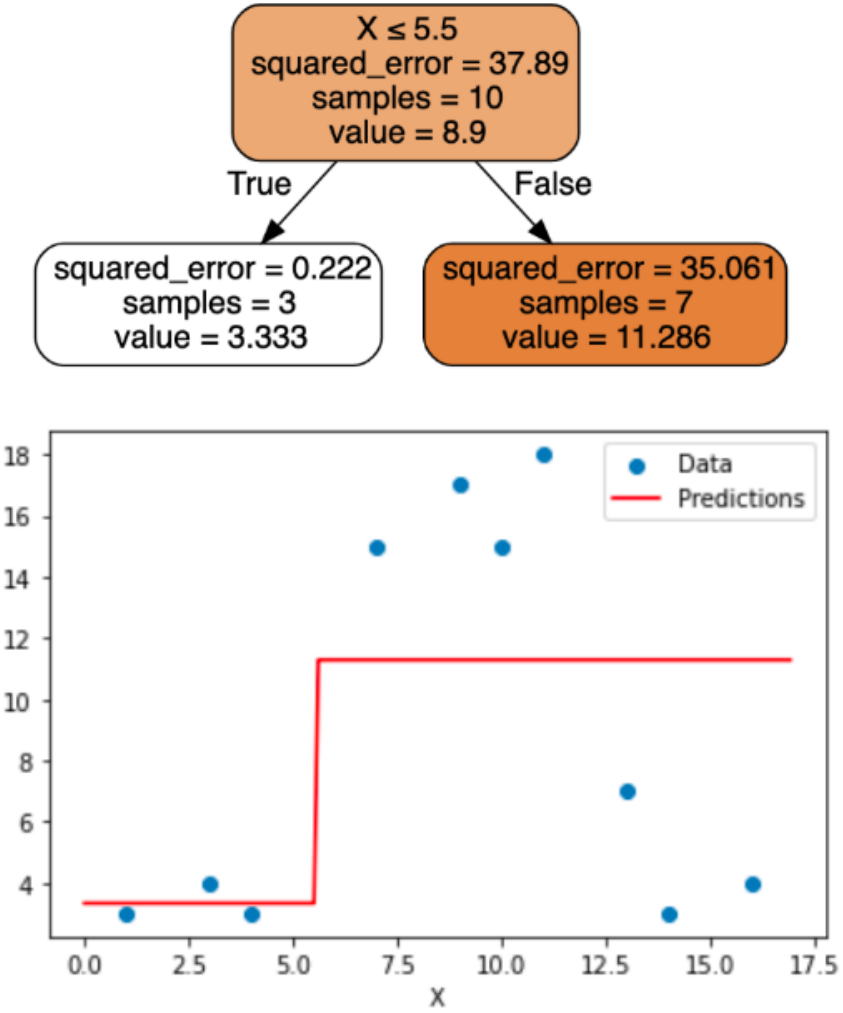
- 주요 규제 하이퍼파라미터 (DecisionTreeClassifier / Regressor)

1) max_depth: 트리의 최대 깊이 제어 (가장 직관적인 규제 방법).
2) min_samples_split: 노드를 분할하기 위한 최소 샘플 수.
3) min_samples_leaf: 리프 노드가 가져야 할 최소 샘플 수.
4) max_leaf_nodes: 생성될 수 있는 리프 노드의 최대 개수.
5) max_features: 각 노드 분할 시 고려할 최대 피처 수.
6) max_로 시작하는 매개변수를 줄이거나, min_으로 시작하는 매개변수를 늘리면 규제가 강화되어 과대적합이 방지된다.

12. 머신 러닝

- 지도 학습을 이용한 회귀(2)

트리 기반 회귀 (Decision Tree Regression)



트리 기반 회귀 (Decision Tree Regression)

- 회귀 모델 동작 원리

  1) 클래스 레이블을 예측하는 대신 연속적인 숫자값(Value)을 예측.

  2) 각 리프 노드에 속한 학습용 데이터들의 평균(Mean)을 해당 영역의 최종 예측값으로 사용

- 분할 기준

  - 평균제곱오차(MSE)를 최소화하는 피처와 임계값을 탐색하여 학습용 데이터를 두 개의 서브셋으로 나눔

트리 기반 회귀 모델 실습

- 데이터 불러오기(보스톤 집값 예측)
- 학습용과 평가용 데이터 생성
- 결정 트리 회귀 모델 생성 함수 호출, 회귀 모델 생성

```
from sklearn.tree import DecisionTreeRegressor
dt = DecisionTreeRegressor(random_state=0)
```

-모델 학습

```
dt.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test = dt.predict(X_test)
```

- 모델 평가

```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import root_mean_squared_error as rmse
# 평가용 데이터에 대한 성능 측정
rmse_dt = rmse(y_test, pred_test)
```

트리 기반 회귀 모델 실습

결정 트리의 불안정성 (Instability)

- 결정 트리의 치명적 한계

1) 축 수직 분할(Axis-aligned Boundaries)의 한계

: 결정 트리는 공간을 '축에 수직'으로만 분할, 만약 실제 데이터의 정답 경계선이 완만한 대각선(곡선) 형태라면, 단일 트리는 이를 표현하기 위해 수많은 '계단 모양'으로 조각조각 잘라야 하며 이 매끄럽지 못한 경계면이 결국 예측 오차를 만든다.

2) 높은 분산(High Variance) 및 민감성:

: 훈련 데이터의 아주 미세한 변화(샘플 추가/삭제)에도 트리 전체 구조가 완전히 재구성될 수 있음

- 해결 방안

: 단일 결정 트리의 불안정성을 극복하기 위해 여러 트리를 결합하는 앙상블(Ensemble: 베깅, 부스팅 등) 기법 활용 (다음 단원에서 학습).

13. 머신 러닝

- 앙상블

앙상블 학습과 투표/평균 기반 예측

- 앙상블 학습(Ensemble Learning)

  : 여러 개의 단일 결정 트리 모델을 연결하여 더 강력한 모델을 만드는 기법



- 종류

  1) Bagging
  2) Boosting
  3) stacking

앙상블 학습과 투표/평균 기반 예측

- 분류 문제: 투표 기반 분류(다수결)

1) 직접 투표 (Hard Voting):

· 각 개별 분류 모델이 예측한 최종 클래스 레이블들의 다수결(Majority Vote)로 최종 클래스 결정

2) 간접 / 소프트 투표 (Soft Voting):

① 각 개별 분류 모델이 출력하는 클래스별 예측 확률의 평균을 산출하여, 가장 높은 확률의 클래스를 선택

② 개별 모델의 예측 확신도(Confidence)를 반영하므로 일반적으로 Hard Voting보다 성능이 더 우수함

- 회귀 문제: 평균 기반 회귀

1) 작동 원리: 회귀 문제에서는 연속형 개별 예측값이 출력되므로 투표(다수결) 적용이 불가능함

2) 집계 방식: 각 개별 회귀 모델들이 예측한 연속값들의 평균을 계산하여 최종 예측값으로 결정

베깅(Bagging)과 페이스팅(Pasting)

- 샘플링 방식에 따른 구분

1) 베깅 (Bagging - Bootstrap Aggregating)

① 학습용 데이터에서 복원 추출 방식으로 중복을 허용하여 서브 데이터 생성

② 각 서브 데이터로 모델을 개별 훈련한 후 결과를 집계(분류: 다수결, 회귀: 평균)

2) 페이스팅 (Pasting)

: 학습용 데이터에서 비복원 추출 방식으로 중복 없이 서브 데이터 생성

랜덤 패치(Random Patches)와 랜덤 서브스페이스(Random Subspaces)

- 특성 무작위 추출 (Feature Sampling)

  : '행' 뿐만 아니라 '특성(열)' 까지 무작위로 추출하여 모델 간 다채로움(Diversity) 확보

- 두 기법의 개념 및 매개변수 비교

  1) 랜덤 패치 기법 (Random Patches Method)

     ① 샘플과 특성을 모두 무작위 추출.

     ② bootstrap=True & max_features < 1.0

  2) 랜덤 서브스페이스 기법 (Random Subspaces Method)

     ① 샘플은 전체를 사용하고, 특성만 무작위 추출.

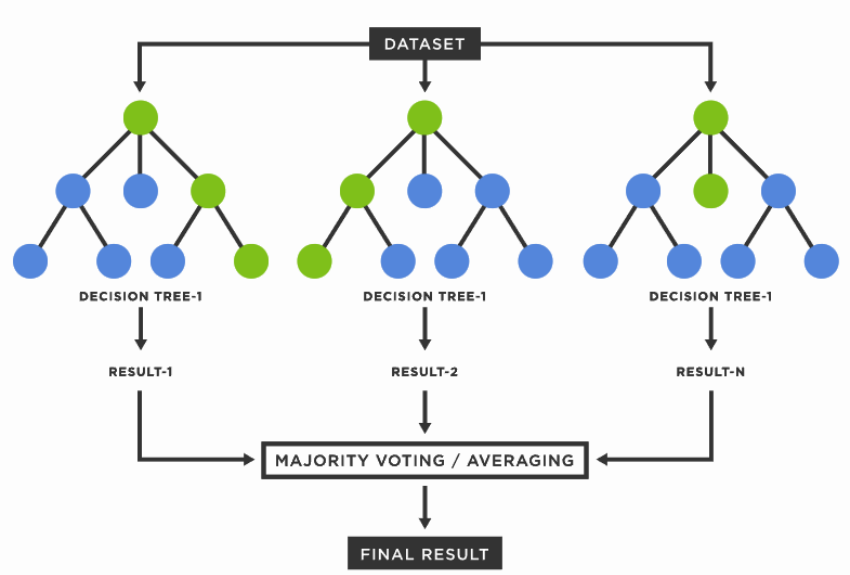
     ② bootstrap=False & max_features < 1.0

  3) 장점: 고차원 데이터에서 일부 강한 특성에 모델이 편향되는 현상을 예방함

랜덤 포레스트(Random Forest)

- 개념: 결정 트리(Decision Tree)를 기본 모델(Base Estimator)로 사용하는 랜덤 패치 기반의 대표적 베깅 앙상블
- 트리의 무작위성 (Feature Subspaces at Nodes):

: 노드를 분할할 때 전체 특성을 검토하지 않고, 무작위로 선택된 특성의 부분집합 $(\sqrt{n_features})$ 중에서 최선의 분할 특성을 선택



랜덤 포레스트(Random Forest) 사용법

- 분류 모델

```
# 필요한 라이브러리 임폴트
from sklearn.ensemble import RandomForestClassifier
# 랜덤 포레스트 분류 모델 생성 함수 호출, 분류 모델 생성
rf = RandomForestClassifier(매개변수)
```

- 회귀 모델

```
# 필요한 라이브러리 임폴트
from sklearn.ensemble import RandomForestRegressor
# 랜덤 포레스트 회귀 모델 생성 함수 호출, 회귀 모델 생성
rf_reg = RandomForestRegressor(매개변수)
```

랜덤 포레스트(Random Forest) 분류 모델 실습

- 데이터 불러오기(당뇨병 건강 지표 데이터)
- 학습용과 평가용 데이터 생성
- RandomForest 분류 모델 생성

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(random_state=0)
```

- 모델 학습

```
rf.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test_rf = rf.predict(X_test))
```

- 모델 평가

```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import accuracy_score
# 평가용 데이터에 대한 성능 측정
acc_rf = accuracy_score(y_test, pred_test_rf)
```

랜덤 포레스트(Random Forest) 분류 모델 실습

랜덤 포레스트(Random Forest) 회귀 모델 실습

- 데이터 불러오기(보스톤 집값 데이터)
- 학습용과 평가용 데이터 생성
- RandomForest 회귀 모델 생성

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(random_state=0)
```

- 모델 학습

```
rf.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test_rf = rf.predict(X_test))
```

- 모델 평가

```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import root_mean_squared_error as rmse
# 평가용 데이터에 대한 성능 측정
rmse_rf = rmse(y_test, pred_test_rf)
```

랜덤 포레스트(Random Forest) 회귀 모델 실습

부스팅(Boosting)과 XGBoost (Extreme Gradient Boosting)

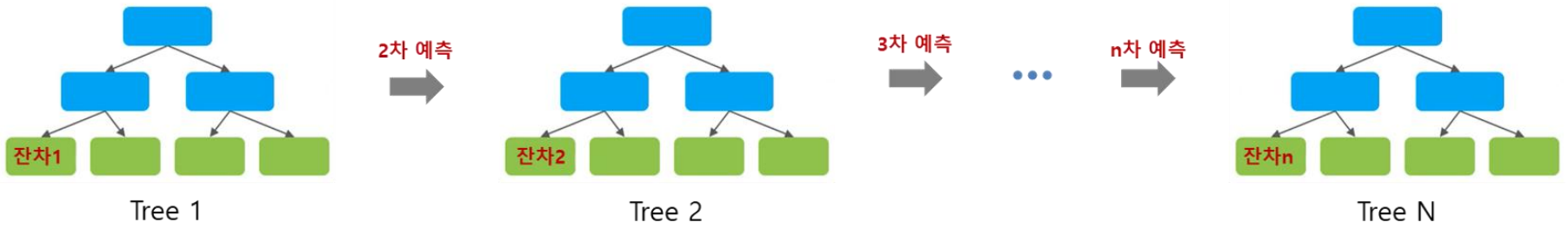
- 부스팅(Boosting)의 핵심 메커니즘

: 순차적 학습 (Sequential Learning): 이전 모델의 오류를 다음 모델이 보완하는 체인 형태의 앙상블 알고리즘

Height (m)	Gender	Age	Weight (kg)
1.8	Male	27	88
1.7	Male	44	68
1.7	Male	58	76
1.5	Female	15	35
1.6	Female	25	54

평균 : 64.2

Height (m)	Gender	Age	Weight (kg)	Residual (1차)	Residual (2차)
1.8	Male	27	88	23.8	21.42
1.7	Male	44	68	3.8	68 – (64.2 + (0.1X7.8)) =
1.7	Male	58	76	11.8	76 - (64.2 + (0.1X7.8)) =
1.5	Female	15	35	-29.2	-26.28
1.6	Female	25	54	-10.2	-9.18



부스팅(Boosting)과 XGBoost (Extreme Gradient Boosting)

- '조각상 다듬기'
  - 초기값: 대략적인 큰 덩어리를 잡는다(첫 번째 트리 예측: 평균값으로 오차 계산)
  - 1단계: 튀어나온 큰 오차 부분을 깎아낸다(두 번째 트리 예측 x 학습률)
  - 2단계: 아직 덜 깎이거나 잘못 깎인 미세한 오차를 보고 조금 더 깎아낸다(세 번째 트리 예측 x 학습률)
  - 이를 수백 번 반복하여 점점 정밀한 조각(완성된 모델)을 만들어낸다.

XGBoost(Extreme Gradient Boosting) 사용법

- 분류 모델

```
# 필요한 라이브러리 설치
pip install xgboost
# 필요한 라이브러리 임폴트
from  xgboost  import XGBClassifer
# XGBoost 분류 모델 생성 함수 호출, 분류 모델 생성
xgb = XGBClassifer(매개변수)
```

- 회귀 모델

```
# 필요한 라이브러리 임폴트
from  xgboost  import XGBRegressor
# XGBoost 회귀 모델 생성 함수 호출, 회귀 모델 생성
xgb_reg = XGBRegressor(매개변수)
```

XGBoost 분류 모델 실습

- 데이터 불러오기(당뇨병 건강 지표 데이터)
- 학습용과 평가용 데이터 생성
- XGBoost 분류 모델 생성

```
from  xgboost  import XGBClassifer
xgb = XGBClassifer(매개변수)
```

- 모델 학습

```
xgb.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test_xgb = xgb.predict(X_test)
```

- 모델 평가

```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import accuracy_score
# 평가용 데이터에 대한 성능 측정
acc_xgb = accuracy_score(y_test, pred_test_xgb)
```

XGBoost 분류 모델 실습

XGBoost 회귀 모델 실습

- 데이터 불러오기(보스톤 집값 데이터)
- 학습용과 평가용 데이터 생성
- XGBoost 회귀 모델 생성

```
from xgboost import XGBRegressor
xgb = XGBRegressor(매개변수)
```

- 모델 학습

```
xgb.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test_xgb = xgb.predict(X_test))
```

- 모델 평가

```
# 모델의 성능을 평가하기 위한 함수 임폴트
from sklearn.metrics import root_mean_squared_error as rmse
# 평가용 데이터에 대한 성능 측정
rmse_xgb = rmse(y_test, pred_test_xgb)
```

XGBoost 회귀 모델 실습

스태킹 (Stacking / Stacked Generalization)

- 스태킹의 기본 원리
  - 학습용 데이터에 대한 개별 모델들의 예측 결과들을 새로운 특성(Feature)으로 만들어 메타 모델(최종 예측 모델)을 학습시키는 앙상블 기법
- 스태킹 학습 2단계 구조

1) 1단계 (Base Models): 서로 다른 알고리즘(Logistic Regression, Decision Tree 등)으로 개별 모델 학습 및 예측 수행
2) 2단계 (Meta Model): 1단계 모델들의 예측값들을 입력 데이터(X)로 삼아 최종 메타 모델(예: Logistic Regression / LinearRegression)을 훈련
3) 정리: [기반 모델들의 train] → [출력값 + 정답 결합] → [메타 모델의 train]

모델 A 예측값 (M_A)	모델 B 예측값 (M_B)	모델 C 예측값 (M_C)	실제 집값 (Y)
49,000	51,500	48,000	**50,000**
36,000	34,800	37,000	**35,000**
83,000	86,000	80,000	**85,000**

스태킹 (Stacking / Stacked Generalization)

- 스태킹 예측 2단계 구조

1) 1단계 (Base Models): 서로 다른 알고리즘(Logistic Regression, Decision Tree 등)으로 평가용 데이터에 대한 예측 수행

2) 2단계 (Meta Model): 1단계 모델들의 예측값들을 입력 데이터(X)로 삼아 최종 메타 모델(예: Logistic Regression / LinearRegression) 예측

3) 정리: [기반 모델들의 predict] → [출력값 결합] → [메타 모델의 predict]

모델 A 예측값 (M_A)	모델 B 예측값 (M_B)	모델 C 예측값 (M_C)
49,000	51,500	48,000
36,000	34,800	37,000
83,000	86,000	80,000

스태킹 (Stacking / Stacked Generalization) 실습

- 데이터 불러오기(당뇨병 건강 지표 데이터)
- 학습용과 평가용 데이터 생성
- 1단계: 개별 기본 모델(Base Estimators) 생성

```
base_estimators = [
    ('rf', RandomForestClassifier(random_state=0)),
    ('xgb', XGBClassifier(random_state=0)),
    ('dt', DecisionTreeClassifier(random_state=0))
]
```

- 2단계: 최종 메타 모델(Meta Learner / Blender) 생성

```
final_meta_learner = LogisticRegression(max_iter=1000, random_state=0)
```

스태킹 (Stacking / Stacked Generalization) 실습

- Stacking 모델 생성

```
stack = StackingClassifier(
    estimators=base_estimators,
    final_estimator=final_meta_learner
)
```

- 모델 학습

```
stack.fit(X_train, y_train)
```

- 평가용 데이터를 이용한 예측

```
pred_test_stack = stack.predict(X_test))
```

- 모델 평가

```
from sklearn.metrics import accuracy_score
acc_stack = accuracy_score(y_test, pred_test_stack)
```

스태킹 (Stacking / Stacked Generalization) 실습

모델 선택 기준

[1단계] 비즈니스 제약 확인
- 설명 가능성 필수인가?
- 추세 예측인가?

[2단계] 후보 모델 일괄 실험
- 선형/로지스틱, SVM, 단일 트리
- 랜덤포레스트, 부스팅, 스태킹

[3단계] 정량 지표 평가 & 선택
- 분류: F1-score
- 회귀: RMSE

14. 머신 러닝

- 차원 축소

차원의 저주 (Curse of Dimensionality)

- 차원의 저주 개념

  : 데이터의 차원(특성 수)이 증가함에 따라 공간의 부피가 수식적으로 급격히 증가하여, 데이터 밀도가 극도로 희박(Sparse)해지는 현상

- 주요 문제점

  1) 과대적합(Overfitting) 위험 극대화: 샘플에 비해 차원이 많아지면 모델이 잡음까지 과도하게 학습함

  2) 거리 측정의 무의미화: 고차원에서는 거의 모든 샘플 간의 거리가 비슷해져, 유클리드 거리 기반 알고리즘(SVM 등)의 성능이 저하됨

- 해결 방안

  1) 학습용 데이터 밀도 올리기: 차원이 늘어나는 만큼 엄청난 양의 학습용 데이터를 추가 (현실적으로 불가능)

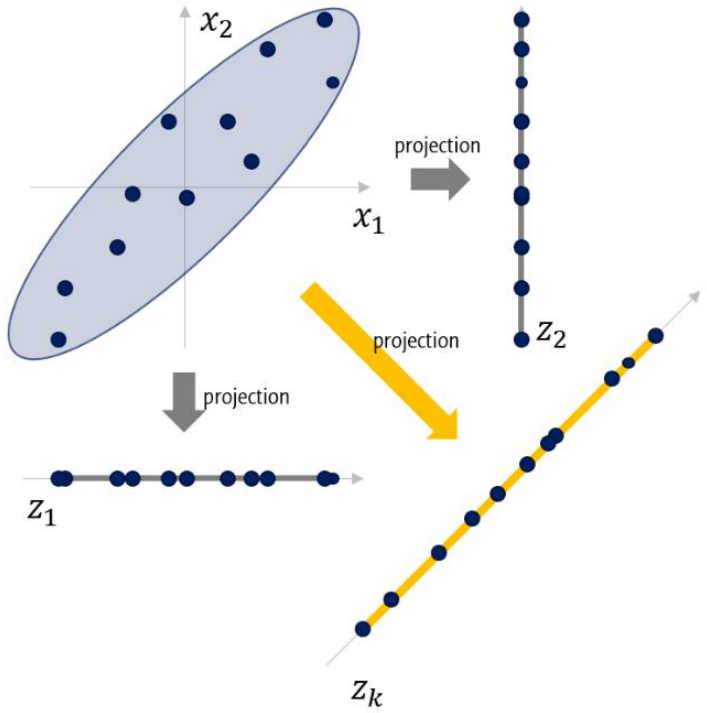
  2) 차원 축소(Dimensionality Reduction)

차원 축소를 위한 알고리즘

- 선형 축소 알고리즘

  1) 고차원 공간에 있는 데이터 점들을 더 낮은 차원의 평평한 부분 공간에 수직으로 투영하는 방식

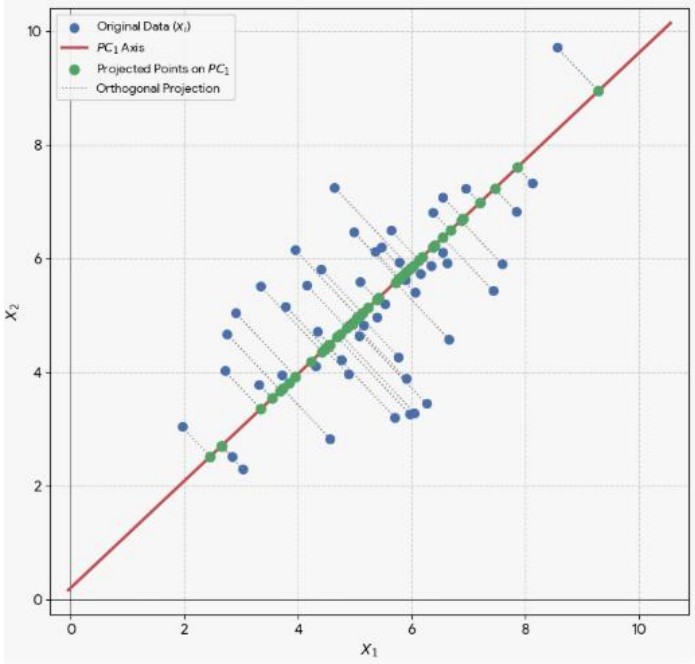
  2) 대표 알고리즘: PCA (Principal Component Analysis)



주성분 분석 (PCA - Principal Component Analysis)

- PCA의 핵심 원리

1) 수많은 데이터 사이에서 '데이터가 가장 넓게 퍼져서 데이터의 원래 형태를 가장 잘 나타내는 최고의 각도'를 찾아내고, 그 각도로 데이터를 투영시켜 영리하게 차원을 줄이는 알고리즘



2) 첫 번째 주성분(PC1): 데이터의 분포(분산)가 가장 넓게 퍼지는 축

3) 두 번째 주성분(PC2): PC1과 직교하면서 데이터의 분포(분산)가 가장 넓게 퍼지는 축

주성분 분석 (PCA - Principal Component Analysis)

- PCA의 주성분 개수 선택 방법
  1) 시각화 목표: 2개(2차원)
  2) 원본 데이터 정보의 80~90% 이상을 유지하도록 주성분 개수 d를 선택

주성분 분석 (PCA - Principal Component Analysis) 실습

- 고차원(64차원) 손글씨 이미지 데이터 생성
- PCA 모델 생성 및 데이터 변환
  1) 시각화 목적: 64차원 -> 2차원으로 차원 축소
     ① 모델 생성
     ② 데이터 변환 및 결과 시각화
  2) 원본 정보의 95%를 유지하는 차원 축소
     ① 모델 생성
     ② 데이터 변환

15. 머신 러닝

- AutoML

AutoML(Automated Machine Learning)

- 개념: 머신러닝 모델을 구축할 때 필요한 반복적이고 복잡한 과정(전처리, 모델 선택, 성능 고도화)을 자동화하는 기술
- 기능: 데이터 전처리, 모델 선택, 성능 고도화, 모델 결합 등 복잡한 과정을 자동화
- 필요성: 고성능 머신러닝 모델 개발 시간을 획기적으로 단축하여 데이터 분석가가 '비즈니스 가치 창출'에 더 집중하게 함
- 예: AutoGluon (Amazon 개발 오픈소스 라이브러리)

AutoGluon

- 파이썬 기반의 오픈 소스 머신러닝 라이브러리
- 기능: 단 몇 줄의 코드로 전문가 수준(SOTA)의 성능 구현

AutoGluon

구분	**전통적 방식 (Manual)**	**AutoGluon (AutoML)**
모델 탐색	모델별 개별 코드 작성 및 파이프라인 구축	한 줄의 코드로 수십 개 모델 동시 학습 및 비교
데이터 전처리	수동 처리 (인코딩, 누락 데이터, 스케일링 직접 구현)	**기초 전처리 자동화** *(단, 도메인 파생 변수 생성과 노이즈 정제는 제외)*
성능 고도화	GridSearchCV 등을 통한 개별 하이퍼파라미터 수동 최적화	**모델별 하이퍼파라미터 최적화 자동** 및 **다층 스태킹 앙상블 파이프라인 자동화**
성능 (결과)	분석가의 숙련도와 시간 투자에 따라 성능 편차 큼	짧은 시간 내에 **전문가 수준(SOTA)** 성능 구현

AutoGluon

- AutoGluon 3-Step 워크플로우
  1) TabularPredictor 클래스: 모델 생성
  2) fit 함수: 학습 및 최적화, best 모델 생성
  3) evaluate 함수: best 모델 성능 평가(예측 & 평가)

AutoGluon

- 사용법

```
# 설치:  pip install -U "autogluon.tabular[all]>=1.6.0"


# [준비 단계] 데이터 분할: sklearn 라이브러리 train_test_split 함수 사용
train_df, test_df = train_test_split(df, test_size=0.2, random_state=0, stratify=df['Churn'])

# 3-Step 워크플로우 적용

## 1. 모델 생성
predictor = TabularPredictor(label='Churn', eval_metric='accuracy', path='원하는 위치 지정')
```

AutoGluon

```
## 2. 학습 및 최적화, best 모델 생성
predictor.fit(
  train_data=train_df,
  presets='best_quality',
  num_gpus=0,                  # CPU만 사용
  dynamic_stacking=False,    # 사전 탐색(DyStack) False
  num_stack_levels=1,
  time_limit=300
))

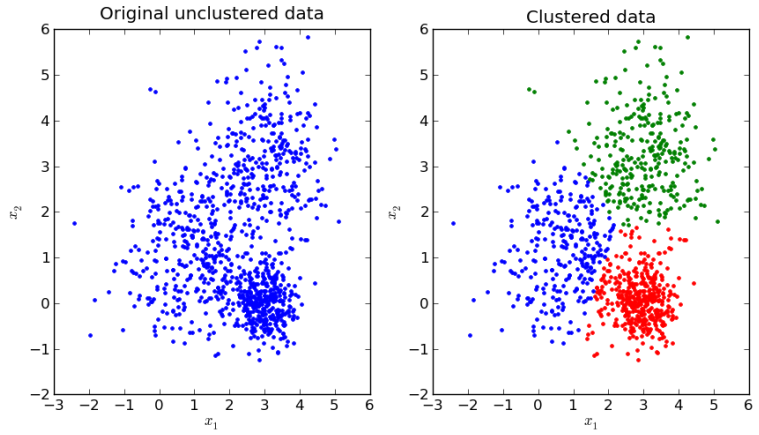
## 3. best 모델 성능 평가
performance = predictor.evaluate(test_data)
```

16. 머신 러닝

- 비지도 학습

군집화(clustering)

- 유사한 속성들을 갖는 데이터를 묶어 전체 데이터를 몇 개의 군집(그룹)으로 나누는 것



- 군집화 기준

1) 군집 간 분산 (inter-cluster variance) : 최대화, 집단 간의 이질성 최대

2) 군집 내 분산 (inner-cluster variance) : 최소화, 집단 내의 유사성 최대

- 분류(classification)과 군집화(clustering) 비교

1) 분류 : 사전에 정의된 범주가 있는(labeled) 데이터를 학습 후 미지의 데이터에 대해서 범주를 예측

2) 군집화 : 사전에 정의된 범주가 없는(unlabeled) 데이터로부터 최적의 군집을 찾는 것

군집화(clustering) 알고리즘 : K-Means

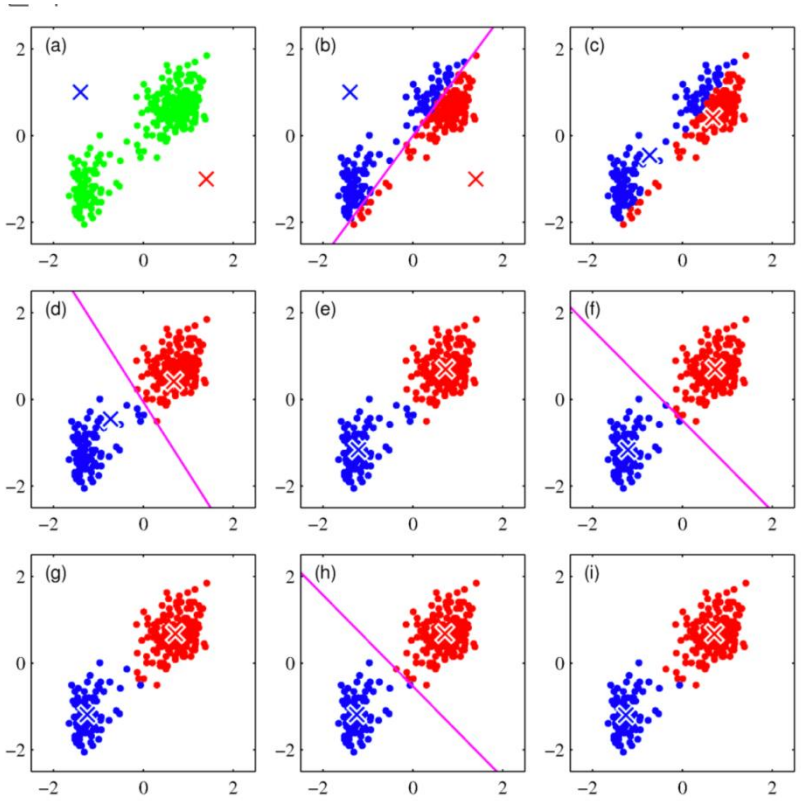
- 군집화에서 가장 일반적인 알고리즘

1) 임의의 K의 개수 만큼 클러스터를 묶는 방식

2) 주어진 데이터들 사이의 유클리디안 거리를 측정하여 군집화

3) 원리

- 초기 중심점 설정
- 각 데이터를 가장 가까운 중심점에 할당, 초기 클러스터 생성
- 클러스터에 할당된 데이터 간 거리의 평균을 계산
- 중심점을 다시 지정 → 중심점 이동
- 각 데이터를 이동된 중심점 기준으로 가장 가까운 중심점에 할당, 새로
- 새로운 클러스터에 할당된 데이터 간 거리의 평균을 계산
- 중심점을 다시 지정 → 중심점 이동
- 중심점 변경과 이동이 없으면 종료



군집화(clustering) 알고리즘 : K-Means

- 사용법

```
# 필요한 라이브러리 임폴트
from sklearn.cluster import KMeans

# Kmeans 모델 생성 함수 호출,  모델 객체 생성
model = KMeans(n_clusters=k, random_state)

# Kmeans 모델 학습 및 군집화의 결과 확인
model.fit_predict(data)
```

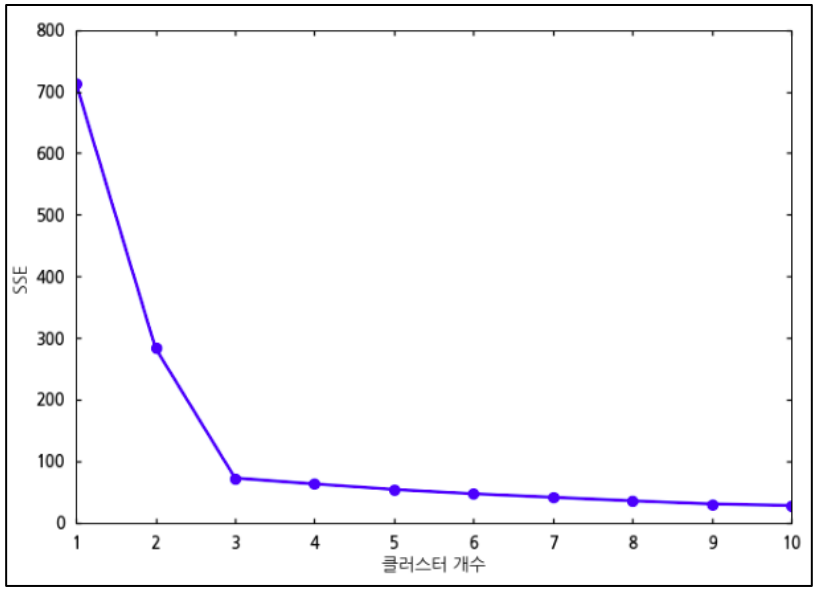
군집화(clustering) 알고리즘 : K-Means

- 최적의 클러스터 개수 설정 : 엘보우(elbow) method

1) 클러스터내 오차제곱합(SSE)이 최소가 되도록 클러스터의 중심을 설정하는 방법

2) 클러스터의 개수를 1에서 n으로 증가 → SSE의 변화를 관찰

3) SSE의 변화가 가장 크게 감소하는 구간 → 최적의 개수



K-Means 실습

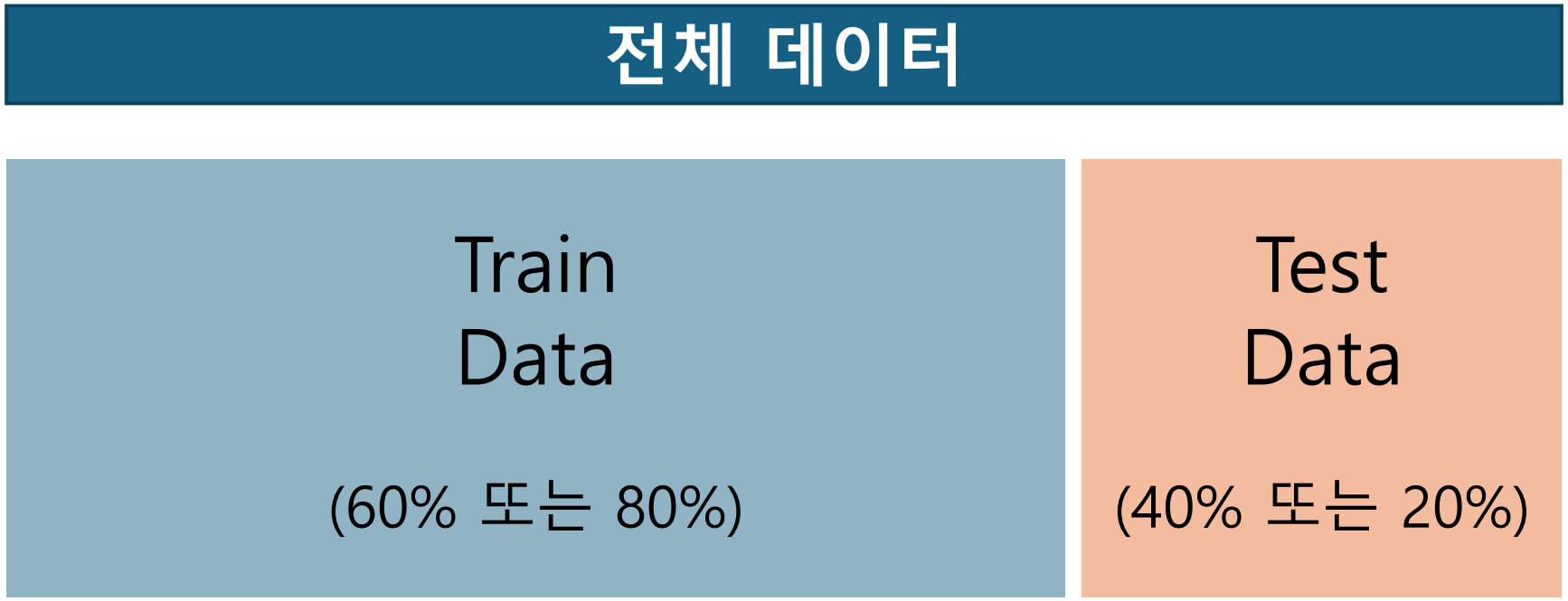
- 붓꽃 데이터

17. 머신 러닝

- 모델 성능 개선

모델 평가 방법의 개선

- 지양해야 할 평가 방법



모델 평가 방법의 개선

- K-fold cross-validation(교차 검증)

  : "운 좋게 맞춘 건가, 진짜 실력인가?"를 판별하는 표준 기법



1. Train 데이터를 k개의 그룹으로 나눈다.
2. K-1개의 그룹을 학습에 사용한다.
3. 나머지 1개의 그룹을 이용해서 평가를 수행한다.
4. 2번, 3번 과정을 k번 반복한다.
5. 모든 결과의 평균을 구한다.

모델 평가 방법의 개선

- 실무에서의 적용 방식 정리

  1) 일반 머신러닝 (Tabular Data, Scikit-learn 기반): K-fold cross-validation(교차 검증)

  2) 딥러닝 (이미지, 자연어, 거대언어모델 LLM): Hold-out 검증

     ① 전체 데이터를 학습용(Train), 검증용(Validation), 평가용(Test)으로 무작위 분할하여 모델의 성능을 평가

     ② 이유: 비용과 시간 때문

     ③ 전체 데이터가 충분히 크다면, 무작위로 딱 한 번만 8:2나 9:1로 쪼개도(n_splits=1) 학습용 데이터와 검증 데이터가 전체 데이터의 분포를 충분히 대변한다.

GridSerchCV를 이용한 모델 성능 고도화

- sklearn 라이브러리에서 제공하는 모델 성능 극대화를 위한 최적의 하이퍼파라미터 탐색 및 교차 검증 도구
- 사용자: 성능 극대화 대상 모델, 최적화 대상 하이퍼파라미터와 탐색 값 설정

```
params = {
    'max_depth':[3,4,5,6,7,8,9,10],
    'min_samples_leaf':[2,3,4,5]
}
```

- GridSearchCV는 사용자가 제시한 하이퍼파라미터와 탐색 범위 내의 모든 조합에 대해 학습과 평가를 수행
- 가장 성능이 좋은 최적의 하이퍼파라미터 값을 도출한다
- 최적화 모델 완성

GridSerchCV를 이용한 모델 성능 고도화

- GirdSerchCV 사용법

```
# 필요한 함수 임폴트
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# 성능 고도화 대상 모델 생성
dt = DecisionTreeClassifier(random_state=0)

# 최적화 할 매개 변수와 탐색의 조건 설정 -> key : value -> python dict
params = {
    'max_depth':[3,4,5,6,7,8,9,10],
    'min_samples_leaf':[2,3,4,5]
}

# GridSearchCV 탐색 도구 호출, 도구 생성
grid_dt = GridSearchCV(
    estimator=dt,
    param_grid=params,
    scoring='accuracy',
    cv=5
)

# 탐색 도구 학습 및 평가
grid_dt.fit(X_train, y_train)
```

```
# beat 모델 추출
best_dt = grid_dt.best_estimator_

# 평가용 데이터에 대한 예측
pred_test_dt = best_dt.predict(X_test)

# 평가용 데이터에 대한 성능 평가
acc_dt = accuracy_score(y_test, pred_test_dt)
```

GridSerchCV를 이용한 분류 모델 성능 고도화 실습

GridSerchCV를 이용한 회귀 모델 성능 고도화 실습

RandomizedSearchCV를 이용한 모델 성능 고도화

- sklearn 라이브러리에서 제공하는 모델 성능 극대화를 위한 최적의 하이퍼파라미터 탐색 및 교차 검증 도구
- 사용자: 성능 극대화 대상 모델, 최적화 대상 하이퍼파라미터와 탐색 값 설정

```
param_distributions = {
    'max_depth': range(3,21),
    'min_samples_leaf': range(2, 11)
}
```

- RandomizedSearchCV는 사용자가 제시한 하이퍼파라미터와 탐색 범위 내의 모든 조합에 대해 설정한 횟수(n_iter)만큼 무작위로 샘플링하여 탐색 수행
- 가장 성능이 좋은 최적의 하이퍼파라미터 값을 도출한다
- 최적화 모델 완성

RandomizedSearchCV를 이용한 모델 성능 고도화

- RandomizedSearchCV 사용법

```
# 필요한 함수 임폴트
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

# 성능 고도화 대상 모델 생성
dt = DecisionTreeClassifier(random_state=0)

# 최적화 할 매개 변수와 탐색의 조건 설정
param_distributions = {
    'max_depth': range(3,21),
    'min_samples_leaf': range(2, 11)
}

# RandomizedSearchCV 탐색 도구 호출, 도구 생성
random_dt = RandomizedSearchCV(
    estimator=dt,
    param_distributions=param_distributions,
    n_iter=20,
    scoring='accuracy',
    cv=5,
    random_state=0
)

# 탐색 도구 학습 및 평가
random_dt.fit(X_train, y_train)
```

```
# beat 모델 추출
best_dt = random_dt.best_estimator_

# 평가용 데이터에 대한 예측
pred_test_dt = best_dt.predict(X_test)

# 평가용 데이터에 대한 성능 평가
acc_dt = accuracy_score(y_test, pred_test_dt)
```

RandomizedSearchCV를 이용한 분류 모델 성능 고도화 실습

RandomizedSearchCV를 이용한 회귀 모델 성능 고도화 실습


MISSION:
COMPLETE